

6. 動態規劃

「每一個 DP 背後都有一個 dag，DP 需要 dag 指引方向，如同視障者需要 dog。」
(Dynamic programming, Bottom-up)

「設計 DP 以尋找遞迴式開始，以避免遞迴來完成，除非，除非你準備小抄。」
(Dynamic programming, Top-down memoization)

這一章介紹動態規劃 (Dynamic Programming, DP)。DP 名字中 Programming 的意思是指「以表格紀錄」，而非現在常用的「寫程式」，所以 DP 的意思是以變動的表格來求解的方法。DP 的應用很廣，變化很多，在學術界也發展的很早，在程式競賽也用的非常多，並且有很多困難而精妙的 (優化) 方法。這本教材只限於一些常用的典型的題型，此外樹與圖的 DP 將放在後續討論樹的專章中，而不在本章中。

6.1. 基本原理

6.1.1. 基本思維與步驟

DP 與分治有個相同之處，都是將問題劃分成子問題，再由子問題的解合併成最後的解，其中子問題是指相同問題比較小的輸入資料。所以，設計 DP 的方法從找出遞迴式開始，設計 DP 的演算法通常包含下面幾個步驟：

1. 定義子問題。
2. 找出問題與子問題之間的 (遞迴) 關係。
3. 找出子問題的計算順序來避免以遞迴的方式進行計算。

如果沒有進行第三個步驟，而只是以遞迴的方式寫程式，就會變成與第一章相同的純遞迴方式，純遞迴往往會遭遇到非常大的效率問題，所以也可以說 DP 就是要改善純遞迴的效率問題的技術。我們先介紹兩個簡單的問題，以問題來舉例說明會比較清楚。

- 問題一：小朋友上樓梯，每步走一階或兩階，如果走到第 n 階有 $f(n)$ 種走法，輸入 n ，計算 $f(n)$ 。例如 $n=3$ 時，走法有 $1+1+1=3$ 或 $1+2=3$ 或 $2+1=3$ ，一共 3 種， $f(n)=3$ 。

我們的問題是計算 $f(n)$ 。定義子問題：對 $i < n$ ， $f(i)$ 表示走到第 i 階的走法數。接著要找出 $f(n)$ 與子問題的遞迴關係，我們雖然沒辦法立刻寫出 $f(n)$ 是多少，但是我們只

要去想，走到第 n 階的最後一步一定是走了一階或兩階（沒有其他的可能），而最後一步是一階與兩階的走法必然不同（沒有重複多算）。最後走一階的走法數就是抵達第 $n-1$ 的走法數 $f(n-1)$ ，而最後一步走兩階的走法數就是抵達第 $n-2$ 階的走法數 $f(n-2)$ ，因此，我們得到遞迴關係式 $f(n) = f(n-1) + f(n-2)$ ，但是不要忘了以上推論是對於 $n > 2$ 時的狀況，遞迴的初始條件 $f(1)$ 與 $f(2)$ 要另外找，不難想一下就可以知道，所以完整的遞迴關係是：

$$f(n) = \begin{cases} f(n-1) + f(n-2) & \text{if } n > 2 \\ n & \text{otherwise} \end{cases}$$

如果直接依照遞迴寫程式，以下是這樣簡單的程式。

```
//ch6 stair, recursion
#include <stdio>
long long stair(int n) {
    if (n<3) return n;
    return stair(n-1)+stair(n-2);
}
int main() {
    int n;
    scanf("%d", &n);
    printf("%lld\n", stair(n));
    return 0;
}
```

這個程式可以跑，只要你不要輸入太大的數字。如果輸入 40 大概會看到螢幕停頓一下就跑出來，輸入 60 的話肯定可以去喝杯咖啡再看看跑出來沒；如果輸入 100，跑到你死了也跑不出來；如果輸入 200，那到地球毀滅也跑不出來。原因何在？這是一個純遞迴的程式，一個遞迴呼叫兩個，兩個就會呼叫四個，雖然 $f(n-1)$ 與 $f(n-2)$ 並不一樣，但它會接近 2 的 n 次方，確切的時間複雜度大概是 $O(1.6^n)$ 。

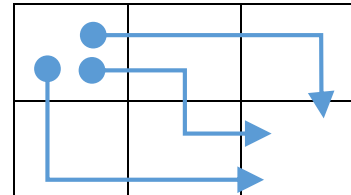
我們來把它改成 DP，所以要進行第三步驟：找出子問題的計算順序來避免以遞迴的方式進行計算。什麼樣的計算順序會讓我們可以不需要遞迴呢？原則上「**只要讓遞迴式等號右邊出現的在左邊的之前先算，並且用表格把算過的記下來，就可以不需要遞迴了。**」以這一題為例，遞迴式 $f(n) = f(n-1) + f(n-2)$ ，如果 n 從小往大算，算到 $f(n)$ 的時候， $f(n-1)$ 與 $f(n-2)$ 都已經算過了，所以只要當初算完的時候有記下來，那就可以直接從表格中取出 $f(n-1)$ 與 $f(n-2)$ 來相加得到 $f(n)$ ，完全不需要用到遞迴。那麼如何用表格紀錄呢？這一題我們可以開一個陣列 $F[]$ ，以 $F[i]$ 記錄 $f(i)$ 的值，程式就變成一個簡單的迴圈，沿著我們的計算順序 (n 從小到大)，逐步從表格中取出值，計算後存入表格，程式如下：

```
//ch6 stair, dp
#include <cstdio>
int main() {
    long long F[10000];
    int n;
    scanf("%d", &n);
    F[1]=1, F[2]=2;
    for (int i=3; i<=n; i++)
        F[i]=F[i-1]+F[i-2];
    printf("%lld\n", F[n]);
    return 0;
}
```

這個程式跑得很快，不過數字太大會溢位就是了，一般在考這一類題目的時候都要求輸出模某數的餘數。我們完成了一個 DP 算法的設計，看起來並不是太難。請再回想一下剛才走過的思維步驟：定義子問題、找遞迴關係、最後找計算順序與定義表格來避免遞迴。所以 DP 從遞迴開始，但以去除遞迴來完成，看來似乎有點難，但以這題來說又很簡單。其實，對於大部分的題目，尋找計算順序與表格都很簡單，最常出現的計算順序就是像這一題：由小到大，而表格就是用遞迴的參數來直接定義。真正難的是遞迴關係式的尋找。

我們再來看一個例子。

- 問題二：有一個 $m \times n$ 的矩形方格，從左上角的格子出發，要到達右下角的格子，每一步只能向右或向下移動一格，若不同的路徑有 $g(m, n)$ 條，輸入 m 與 n ，計算 $g(m, n)$ 。例如 $g(2, 3)=3$ ，如右圖。



我們的問題是計算 $g(m, n)$ 。定義子問題：以 $g(i, j)$ 表示出發點走到 (i, j) 的走法數，以矩陣列與行的方式定義位置，出發點為 $(1, 1)$ ，終點是 (m, n) 。接著要找出 $g(i, j)$ 與子問題的遞迴關係，同樣的，我們不必立刻寫出 $g(i, j)$ 是多少，只要去想，最後一步一定向右或向下，也就是前一個位置一定是 $(i, j-1)$ 或 $(i-1, j)$ ，而且這兩類走法必然不同（沒有重複多算）。根據定義，我們可以得到

$g(i, j) = g(i, j-1) + g(i-1, j)$ for $i > 1$ and $j > 1$ ；初始條件為

$g(i, j) = 1$ for $i = 1$ or $j = 1$ 。得到遞迴式之後直接寫遞迴程式就是純遞迴，程式如下，再一次，遞迴的程式非常沒有效率：

```
//ch6 grid, recursion
#include <cstdio>
long long grid(int m, int n) {
    if (m==1 || n==1) return 1;
}
```

```

    return grid(m, n-1)+grid(m-1, n);
}
int main() {
    int m,n;
    scanf("%d%d", &m, &n);
    printf("%lld\n", grid(m,n));
    return 0;
}

```

要把純遞迴改成 DP，就要找計算順序，讓遞迴關係式右邊的出現在左邊的之前，本題的遞迴式為： $g(i, j) = g(i, j-1) + g(i-1, j)$ ，簡單觀察可以得知，只要 i 與 j 都從小到大就可以滿足需要，所以我們可以採取「由上而下，由左而右」的順序，當然也可以「由左而右，由上而下」，甚至可以沿著 $i+j$ 由小而大的斜線的順序，有點自找苦吃就是了。至於表格，最直覺的方式就是將 $g(i, j)$ 記錄在一個二維陣列 $G[i][j]$ 中。以下是程式，跑得很快，時間複雜度是 $O(mn)$ ，而遞迴版本的複雜度則是組合數 $C(m+n-2, m-1)$ ，因為這一題的答案其實有數學解，是排列組合的題目。請不需要擔心數學解這件事，通常程式考試與競賽都不出有數學解的題目，以這題來說，只要將某些格子變成禁止通行或者做一些變化，就難以用數學的方式求解了，畢竟程式考試主要考的是程式技巧。

```

//ch6 grid, DP
#include <stdio>
int main() {
    long long G[50][50];
    int m,n;
    scanf("%d%d", &m, &n);
    for (int i=1; i<=m; i++)
        G[i][1]=1;
    for (int j=1; j<=n; j++)
        G[1][j]=1;
    for (int i=2; i<=m; i++)
        for (int j=2; j<=n; j++)
            G[i][j] = G[i-1][j] + G[i][j-1];
    printf("%lld\n", G[m][n]);
    return 0;
}

```

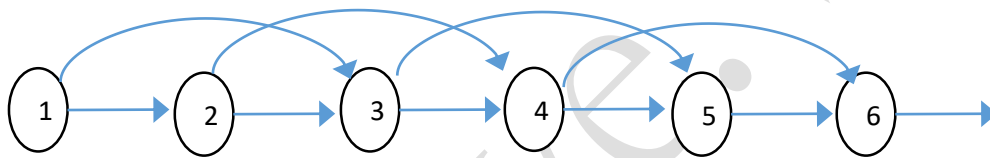
6.1.2. 狀態轉移

DP 子問題的遞迴關係式也有人稱為「狀態轉移」，這個名詞大概是從有限狀態機（數位電路設計）來的。子問題可以看成一個狀態，目前的狀態是根據之前的那些狀態，這樣

的轉換關係就稱作狀態轉移，設計 DP 時，最重要的就是找出狀態轉移。之前我們將子問題看成「部分解」，所以是在找部分解與更小的解之間的關係，這兩個講法只是描述的方式不一樣，其實是同一回事，有的時候這樣想比較方便，有的時候那樣想比較直覺。

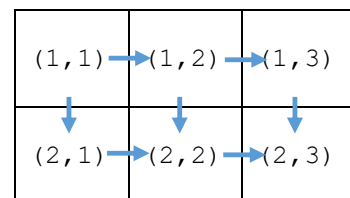
雖然我們要在下一章會討論圖 (graph)，但提到狀態轉移，用圖的術語比較好說明。我們說狀態 A 是狀態 B 的前置狀態 (predecessor)，如果狀態 B 是由 A 轉移過來的，一個狀態可能有多個前置狀態，通常一個狀態的值是由它的前置狀態的值做某些計算得到。如果我們把每個狀態想像每一個點，若 A 是 B 的前置狀態，則由 A 劃一個箭頭指到 B，我們會得到一個所謂的圖 (graph)，這個圖就是這個 DP 的狀態轉換圖，在圖論上來說，它必然是個有方向但沒有環路的圖 (directed acyclic graph, dag)，所謂環路是指從某點出發，沿著箭頭前進，最後又走到出發點。遞迴關係不會無限遞迴，所以必然沒有環路。我們來看看前面兩個簡單的問題的狀態轉移。

小朋友上樓梯問題的子問題是走到第 i 階，關係式 $f(n) = f(n-1) + f(n-2)$ ，所以狀態圖如下，也就是每個狀態由它的前兩個狀態轉移而來。



第二個問題是方格圖的問題，遞迴式是：

$g(m, n) = g(m-1, n) + g(m, n-1)$ ，每一個格子是一個點 (狀態)，前置狀態是他的左方與上方。



對於一個 dag，我們一定可以將它的所有點排成某個順序，在此順序中，每個箭頭都由前往後指，也就是說，每個點的前置點都在它之前，這樣的順序稱為拓樸順序 (topological sort)，我們的 DP 就是要沿著一個拓樸順序來進行。有件事情要提醒，本節說明的狀態轉移以及 dag 與拓樸順序，是方便思考用的，在 DP 解題時通常並不需要把它建出來，也就是建在心中就可以了，不必建在程式中。我們在第三章的 P-3-1 樹的高度的問題時，曾經講過利用 queue 找樹的 bottom-up 順序，那就是個拓樸順序，而那一題確實就是樹的 DP。

6.1.3. 分類與複雜度

題目的分類往往有很多種，主要看目的何在而分類，這裡我們以便於理解與學習來分類。一個 DP 如果狀態有 $O(n^x)$ 而轉移式涉及 $O(n^y)$ 個狀態，一般可稱為 $x\text{DyD}$ 的 DP，例如小朋友上樓梯是 1D0D 的，因為有 n 個要算，每次算 $f(i)$ 只涉及 2 個 $f(i-1)$ 與 $f(i-2)$ 。

2)。方格路徑的問題則是 2D0D，因為有 n^2 (假設 $m=n$) 個要算，每次只需要 2 個 (左方與上方)。當然也有一些 DP 不在這個分類中。

一個 $xDyD$ 的 DP，如果沒有優化，很明顯的，時間複雜度就是 $O(n^{x+y})$ ，一般來說若 $y=0$ 時比較簡單，因為遞迴式單純，而且遞迴式找出來時，程式就已經差不多寫完了，因為套個迴圈依序計算就是了。 $y=1$ 的 DP 題目通常存在某種優化或需要某些資料結構幫忙，所以會比較難。下一節開始的題目分析，我們的會先介紹 $y=0$ ，然後再介紹 $y=1$ 的狀況。

6.1.4. Top-down memoization

前面講到設計 DP 算法的步驟，基本上要先找出遞迴式，然後找出計算順序來避免遞迴。這算是標準的 DP，也稱為 Bottom-up 的 DP。另外有一種實現 DP 的方式是不找計算順序，直接依照遞迴式寫遞迴，但是因為一定要避免遞迴的重複計算，所以採取了一些步驟，這種實現的方式稱為 top-down memoization 的 DP，其中 memoization 字典上應該沒有這個字，大概是為了這個技術自己創造的，這個字是由 memo (備忘錄，便籤小紙條) 變來的，這個方法基本上應該是打比賽或重實作的人上發展出來的 (偷懶) 方法，因為從理論的角度，除了程式比較好寫之外，這麼做沒有好處，但考試與比賽還真的就希望比較好寫。

用比較俏皮的講法，這個方法就是再找到遞迴式之後，直接寫成遞迴版本的程式，但要加上三個動作：

1. 開一個表格當作小抄，用來記錄計算過的結果，表格初值設為某個不可能的值，用以辨識是否曾經算過。
2. 在遞迴之前，先偷看一下小抄是否曾經算過，如果小抄上有答案 (已經算過)，則直接回傳答案。
3. 如果小抄上沒有，就遞迴呼叫，但計算完畢回傳前，要把它記在小抄上。

我們來看看前面介紹的兩個問題的遞迴版本如何改成 top-down DP。下面是第一題，首先注意到它就是拿前面的遞迴版本來改的，修改的部分：第 2 行宣告了一個表格 $F[]$ ；在遞迴函數中第 5 行做了一個檢查是否 $F[n] > 0$ 的動作，如果是則直接回傳，這一題的數字都是正數，所以我們可以用 0 來表示沒算過；如果沒算過，第 6 行就真的遞迴呼叫去計算，然後存在表格中在回傳。

```
00 //ch6 stair, top-down DP
01 #include <cstdio>
```

```

02 long long F[100]={0}; // memo, 0 if not computed
03 long long stair(int n) {
04     // if (n<3) return n; not necessary
05     if (F[n]>0) return F[n]; // check memo
06     F[n] = stair(n-1)+stair(n-2); // record to memo
07     return F[n];
08 }
09 int main() {
10     int n;
11     scanf("%d", &n);
12     F[1]=1, F[2]=2;
13     printf("%lld\n", stair(n));
14     return 0;
15 }

```

這個“遞迴”的程式跑得慢不慢呢？基本上跟迴圈版的 DP 跑得一樣神速。我們再來看第二題方格路徑的改法。下面是修改版的程式，其實每一個 top-down 的 DP 幾乎都是長的這個樣子。這裡我們故意用了一些與上一題稍微不同的修改方式，雖然這題也可以用陣列初值為 0 表示沒算過，但我們自己寫陣列的初始化（設為-1），因為有些題目可能有此需要。上一題我們把遞迴終點條件拿掉，替代以在主程式中設定起始點的表格值，這一題我們留著遞迴的終端條件，其實這些差異都不大。這個程式也是跑得跟 DP 版一樣的神速。

```

//ch6 grid, top-down DP
#include <stdio>
long long G[100][100]; // as memo
long long grid(int m, int n) {
    if (m==1 || n==1) return 1;
    if (G[m][n]>=0) return G[m][n]; // check memo
    // record before return
    return G[m][n]=grid(m, n-1)+grid(m-1, n);
}
int main() {
    int m,n;
    scanf("%d%d", &m, &n);
    for (int i=0; i<=m; i++)
        for (int j=0; j<=n; j++)
            G[i][j] = -1; // initial unknown
    printf("%lld\n", grid(m,n));
    return 0;
}

```

Top-down 的 DP 基本上是個偷懶的方法，我們的建議如下：如果計算順序不難找，還是寫迴圈版，不要過分偷懶，因為遞迴還是有成本與限制，尤其某些系統之下（尤其架在視窗系統下），遞迴的深度有一定的限制，超過限制會導致執行階段的錯誤。在計算順序比較不好找或者比較麻煩時，而且系統是允許的狀況下，就採用 top-down 吧！最

常見的情形就是 Tree 的 DP，幾乎參加比賽的選手都寫遞迴的 DFS 版本，因為迴圈版的要找 bottom-up 順序 (如 P-3-1)，比較麻煩。不過這裡還是要建議：兩種都要會，可以偷懶的時候才偷懶。

6.2. 例題與習題

在本節中我們提出一些動態規劃的經典例題與習題，來讓大家更熟悉以 DP 的思維來設計演算法的方式，我們依照 1D0D、2D0D、1D1D、以及 2D1D 與其他共分成四個小節，在下一節中我們在放了少數比較進階的例子。

6.2.1. 1D0D

這一節我們先看一些 1D0D 的 DP。事實上，在上一節的小朋友上樓梯的走法數就是之前出現過的費氏數列，也就是 1D0D 的一種最簡單形式。用陣列搭配迴圈或甚至使用 Top-down 的方法都可以很快的在 $O(n)$ 求解。我們在第二章 Q-2-5 中曾經提出以矩陣快速幂來做到 $O(\log(n))$ 計算費氏數列的第 n 項，事實上就是這一類 1D0D 的優化方式，如果遞迴式改變時，相同的方法也都可以運用。

不過費氏數列那一類基本上沒有輸入資料的遞迴，其實根本存在數學解，而並非訓練演算法的好題目，大部分的題目是會搭配輸入資料的。我們之前用到過的前綴和 (prefix-sum) 其實也可以算是 1D0D 的 DP，因為遞迴式是 $ps[n] = ps[n-1] + a[n]$ ，其中資料放在 $a[]$ 而 $ps[n]$ 是前 n 項的和，另外像類似的 prefix-minimum 也都是類似的題目，這些都太簡單就不列在例題習題中了。第一個我們選的例題是前面用來說明的小朋友上樓梯，但是是有輸入資料的版本。

P-6-1. 小朋友上樓梯最小成本

小朋友玩上樓梯遊戲，每一步可以往上走一階或兩階，開始位置在第 0 階，從第一階開始每階都有一個數字，踩在第 i 階，分數就要扣第 i 階的數字，請問走到第 n 階的最少的扣分是多少。

Time limit: 1 秒

輸入格式：第一行是正整數 n 。第二行有 n 個正整數，依序代表第 1 階開始的數字，數字間以空白隔開。 $n \leq 1e5$ ，每階的數字不超過 $1e4$ 。

輸出：走到第 n 階的最小總扣分。

範例一輸入： 8 2 1 1 7 3 2 9 2	範例一輸出： 9
範例二輸入： 5 1 2 3 1 5	範例二輸出： 8
範例一說明：最小扣分是 $1+1+3+2+2=9$ 。 範例二說明：最小扣分是 $2+1+5=8$ 。	

假設踩在第 i 階的扣分是 $cost[i]$ ，基本的 DP 問題只要寫出遞迴關係就差不多完成了，通常子問題就是從小到大。根據這個原則定義 $c[i]$ 是走到第 i 階的最小扣分，因為最後一步一定是一階或兩階，我們很快可以得到：

$c[i] = cost[i] + \min(c[i-1], c[i-2])$ for $c > 2$ ；初始條件呢？因為扣分沒有負的，我們很簡單可以知道 $c[1]=cost[1]$ 且 $c[2]=cost[2]$ 。

根據找到的遞迴關係，把 $c[i]$ 存放在陣列中，計算的順序就是 i 由小到大計算，簡單的估算知道用 `int` 就夠大，程式如下很簡單，我們不另外開一個陣列 $c[]$ ，直接將計算結果放在原來輸入的陣列。

```
// P-6-1
#include <bits/stdc++.h>
using namespace std;
#define N 100005
int main() {
    int n;
    int cost[N];
    scanf("%d", &n);
    for (int i=1; i<=n; i++)
        scanf("%d", &cost[i]);
    for (int i=3; i<=n; i++)
        cost[i] += min(cost[i-1], cost[i-2]);
    printf("%d\n", cost[n]);
    return 0;
}
```

這一題跟費氏數列一樣，可以只用 $O(1)$ 的空間，因為我們算 $c[i]$ 只需要 $c[i-1]$ 與 $c[i-2]$ ，所以可以只用兩個變數記住前兩個狀態，以下是這樣的範例程式。我們用了一個小技巧讓前兩個初始設為 0，這樣可以不需要單獨寫程式處理 $n < 3$ 的情形。

```
// P-6-1, O(1) space
#include <bits/stdc++.h>
using namespace std;
#define N 100005
int main() {
    int n, cost, p=0, pp=0; // previous and pre-previous
    scanf("%d", &n);
    for (int i=1; i<=n; i++) {
        scanf("%d", &cost);
        cost += min(p, pp);
        pp = p;
        p = cost;
    }
    printf("%d\n", cost);
    return 0;
}
```

P-6-2. 不連續的表演酬勞

楊鐵心帶著義女穆念慈當街頭的武術表演者，他接到許多的邀約，每天均有一場。每一場表演都可以得到某些金額的報酬，但是武術表演很辛苦，無法連續兩天都進行表演，請你寫一支程式協助他決定應該接受那些表演以得到最大的報酬。

Time limit: 1 秒

輸入格式：第一行是正整數 n 。第二行有 n 個非負整數，依序代表第 1 天開始每天邀約報酬，數字間以空白隔開。 $n \leq 1e5$ ，每天酬勞不超過 10000。

輸出：最大可能獲得的總酬勞。

範例一輸入： 5 1 2 3 1 5	範例一輸出： 9
範例二輸入： 8 2 1 1 7 3 2 9 2	範例二輸出： 18

範例一說明：挑選 $1+3+5=9$ 。

範例二說明：挑選 $2+7+9=18$ 。

把每天可以獲得的酬勞放在一個一維陣列 $p[]$ 中，再一次我們將子問題定義為：
 $dp[i]$ 是前 i 天可以獲得的最大報酬。初始條件很容易想， $dp[0]=p[0]$ ，因為沒有選擇， $dp[1]$ 會是多少呢？因為兩天只能選擇其中之一，所以一定選大的， $dp[1]=\max(p[0], p[1])$ 。要如何建構出遞迴關係式呢？我們可以這麼想：把第 i 天選或不選分開考慮，如果第 i 天不選，前 i 天的獲利就是前 $i-1$ 天的獲利，也就是 $dp[i]=dp[i-1]$ ；如果第 i 天要選呢？我們可以獲得 $p[i]$ ，但是第 $i-1$ 就不可選了，因此最大的獲利是 $p[i]+dp[i-2]$ 。在第 i 天選或不選之間挑最大的，我們得到：
 $dp[i] = \max(dp[i-1], p[i]+dp[i-2])$ 。
 程式就很容易了，讓 i 從小算到大就不需要遞迴了。

```
// max independent on array
#include <bits/stdc++.h>
using namespace std;
#define N 100005
int main() {
    int n, p[N];
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d", &p[i]);
    int dp[N]; // dp[i] = the max until i
    dp[0]=p[0], dp[1]=max(p[0], p[1]);
    for (int i=2; i<n; i++)
        dp[i] = max(dp[i-1], dp[i-2]+p[i]);
    printf("%d\n", dp[n-1]);
    return 0;
}
```

下一題跟上一題看起來似乎有點類似。

P-6-3. 最小監控鄰居的成本

有一條長長的大街被劃分成 n 個區段，編號依序為 $1 \sim n$ 。在第 i 個區段設置監控設備的話，需要成本 $c[i]$ ，而可以監控第 $i-1$ 到第 $i+1$ 的區段 (超出範圍可忽略)。現在要選一些區段裝設監控設備，以便每一個區段都可以被監控到，請計算最少的總成本。

Time limit: 1 秒

輸入格式：第一行是正整數 n 。第二行有 n 個非負整數，依序代表第 i 個區段裝設監控設備的成本，數字間以空白隔開。 $2 < n \leq 1e5$ ，每處成本不超過 $1e4$ 。

輸出：最小監控總成本。

範例一輸入： 5 1 2 3 1 5	範例一輸出： 2
範例二輸入： 8 2 1 1 7 3 2 9 2	範例二輸出： 6

範例一說明：挑選 $1+1=2$ 。

範例二說明：挑選 $1+1+2+2=6$ 。

在一維陣列 $c[]$ 中有一群非負整數，我們要挑選總和盡量小，但是每個沒有被挑選的點必須左右至少有一點被挑到。看起來與前一題類似，假設我們以類似的方式定義 $dp[i]$ 為前 i 個的最小成本，寫遞迴式的時候會碰到一點困難：如果 $dp[i-1]$ 的值是有挑選到 $i-1$ 的，那麼第 i 個點已經被監控到， $dp[i]=dp[i-1]$ ；但是如果 $dp[i-1]$ 的值來自沒有挑選到 $i-1$ ，那麼第 i 點變成非挑不可，但我們又不知道 $i-1$ 是否已經有挑選。

改變一下子問題的定義，我們讓 $dp[i]$ 是前 i 個點的最小監控成本而且最後一點選在 i 。那麼，成本 $c[i]$ 需要加進來，前面的點雖然不確定那些點一定要挑，但是不可能第 $i-1$ 、 $i-2$ 與 $i-3$ 都沒有挑選，否則 $i-2$ 就沒被監控了，因此得到：

$dp[i] = c[i] + \min(dp[i-1], dp[i-2], dp[i-3])$ for $i > 2$;

初始條件是 $dp[0]=c[0]$ ， $dp[1]=c[1]$ ， $dp[2]=c[2]+\min(c[0], c[1])$ 。而最後的解呢？不一定出現在 $dp[n-1]$ ，因為也可能是 $dp[n-2]$ 。程式如下，很簡單就不多做解釋。

```
// min dominating cost on array
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define N 100005
int main() {
    int n, c[N];
    scanf("%d", &n);
```

```

for (int i=0; i<n; i++)
    scanf("%d", &c[i]);
int dp[N]; // dp[i] = the min c until i and include i
dp[0]=c[0], dp[1]=c[1];
dp[2]=c[2]+min(dp[0],dp[1]);
// the last one must be at i-3, i-2, or i-1
for (int i=3; i<n; i++)
    dp[i] = c[i] + min(min(dp[i-3],dp[i-2]),dp[i-1]);
printf("%d\n", min(dp[n-2],dp[n-1]));
return 0;
}

```

如果是剛開始接觸學習 DP 的人要留意，別人跟你說遞迴式很容易懂，但如果你自己碰到題目要列遞迴式的時候，往往列不出來或是列錯，所以在學習的過程不能只看別人怎麼解，而要了解思考的方式。

除了多看題目吸取經驗之外，碰到類似的狀況，通常的解決原則就是把子問題加條件，或是將子問題依照不同的條件再進一步分類。這一題你也可以思考將前 i 個解分成兩類： $dp0[i]$ 代表第 i 個沒選， $dp1[i]$ 代表第 i 個有選，分類後會比較容易列式。但是這一題很多人會列錯遞迴式，原因是這題的成本並沒有單調性，並非範圍越大成本越高， $dp1[i]$ 可能會小於 $dp1[i-1]$ 。如何列式我們留給有興趣的人自己想一想，範例程式中有另外一支以這樣的方式設計的，如果你認真看一看，你會發現兩個程式做的事情是一樣的。我們要強調一下，達到解的思考方式每個人未必相同，有人覺得這樣想比較簡單，有人認為那樣做比較直覺，其實是要找到適合自己的思考模式。

下一題是個習題。

Q-6-4. 闖關二選一

某個遊戲要依序闖過 n 個關卡，初始的時候有一個幸運數字，每個關卡有兩個關卡數字，你必須把自己的幸運數字調整為兩個關卡數字之一，才能通過此關卡，調整的成本是你的幸運數字與關卡數字的差值（絕對值）。請計算出最低闖關總成本。

舉例來說，一開始的幸運數字為 1， $n=2$ ，第一關的過關數字為 (5, -5)，第二關的過關數字為 (-3, -2)。要依序通過兩關，假設第一關把幸運數字調整成 5，第二關調整為 -2，則需要成本為 $|1-5| + |5-(-2)| = 11$ 。如果，第一關把幸運數字 -5，第二關調整為 -3，則需要成本為 $|1-(-5)| + |(-5)-(-3)| = 8$ 。你可以看得出來，總共有四種方式過關，其中最小成本是 8。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 t ，代表關卡數以及初始幸運號碼。接下來有 n 行，每行兩個整數，依序代表每一關的過關數字。 $n \leq 1e5$ ，過關數字的絕對值皆不超過 $1e4$ 。

輸出：最小過關成本。

範例一輸入： 2 1 5 -5 -3 -2	範例一輸出： 8
範例二輸入： 3 0 -5 1 -8 3 -7 -7	範例二輸出： 9

Q-6-4 提示：我們以狀態轉移的方式來思考，有 n 個關卡要通過，每個關卡通過的成本跟上一關通過的方式有關，但是跟更早的無關。所以可以想成通過每一關有兩種狀態，達到第 i 關的第 j 種狀態的最低成本是 $cost[i][j]$ 其中 $i \leq n$ 而 $j=0$ 或 1 。列出相鄰兩關的成本計算方式即可。

Q-6-5. 二維最大子矩陣

輸入一個 $m \times n$ 的二維整數矩陣 $A[1:m][1:n]$ ，要找一塊總和最大的連續子矩陣，輸出其總和。以下圖為例，挑選 $A[1:3][2:3]$ 可以獲得最大總和 13。

2	-2	3	3
-6	5	2	-8
3	7	-2	4

Time limit: 1 秒

輸入格式：第一行有兩個正整數 m 與 n 。接下來 m 行每行 n 個整數，代表矩陣由上而下由左而右的內容。 m 與 n 皆不超過 200，矩陣內的數字絕對值皆不超過 $1e4$ 。

輸出：子矩陣的最大可能總和。

範例一輸入： 3 4 2 -2 3 3 -6 5 2 -8 3 7 -2 4	範例一輸出： 13
--	------------------

範例二輸入：	範例二輸出：	
1 6 -2 1 3 -1 4 -5	7	

這一個習題其實是之前的題目變化成二維的形式，給一個提示：想想看 P-4-13。

6.2.2. 2D0D

1D 的 DP，他們的子問題往往都是以一個整數由小到大編號，2D 的 DP 的子問題通常是由一個二維陣列來編號，第一個要介紹的問題非常簡單，他其實是方格棋盤路徑數問題的有輸入資料版本。

P-6-6. 方格棋盤路線

在一個 $m \times n$ 方格棋盤上，每個格子都有一個分數(可正可負)，現在要從左上角的格子走到右下角，每次只能從當時位置移動到右方或下方的格子，請計算出經過路線上的數字的最大可能總和。以下圖為例，最大的總和路線是經過 (2, -2, 5, 7, -5, 4)，總合為 11。

2	-2	3	3
-6	5	2	-8
4	7	-5	4

Time limit: 1 秒

輸入格式：第一行有兩個正整數 m 與 n 。接下來 m 行，每行 n 個整數，代表方格由上而下，由左而右的內容。 m 與 n 皆不超過 200，矩陣內的數字絕對值皆不超過 $1e4$ 。

輸出：最大總和。

範例一輸入：	範例一輸出：
3 4 2 -2 3 3 -6 5 2 -8 3 7 -5 4	11
範例二輸入：	範例二輸出：
2 5	10

1 2 3 1 -5 -2 5 -8 2 1	
---------------------------	--

範例一說明：如題目中說明

範例二說明：先向右走三步，向下一步，再向右一步，得分 $1+2+3+1+2+1=10$ 。

從第一節介紹的狀態轉移就知道，我們可以將到達每一個格子當作一種狀態，因為只能往右或往下移動，所以每一個格子的前置狀態就是他的左方與上方(如果有的話)。因此我們設計子問題的方式就是：以 $f[i][j]$ 當作是走到位置 (i, j) 所能獲得的最大得分，遞迴式為 $f[i][j] = \max(f[i-1][j], f[i][j-1]) + a[i][j]$ ；其中 $a[i][j]$ 是該格子的分數。初始條件呢？因為最上面一行只能從左方走過來，而最左邊一行只能從上方走過來，所以我們可以對這兩個邊界單獨計算，另外一種方式是將超界的最左方與最上方訂為負的無窮大。計算的順序呢？根據原則，遞迴式等號右邊出現的必須在等號左邊出現的先計算，所以每一個格子的左方與上方只要先出現就是可以的順序，最直接的就是由上而下，由左至右。下面是範例程式，我們沒有另外宣告一個陣列放 $f[][]$ ，而是直接把它存在 $a[][]$ 中。時間複雜度很簡單 $O(mn)$ ，因為有這麼多格子要算，每一格只要 $O(1)$ 。

```
// Grid max weight monotonic path
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define N 205
int main() {
    int m,n;
    int a[N][N]={0};
    scanf("%d%d", &m, &n);
    for (int i=0; i<m; i++) for (int j=0; j<n; j++)
        scanf("%d", &a[i][j]);
    // first row and first column
    for (int j=1; j<n; j++)
        a[0][j] += a[0][j-1];
    for (int i=1; i<m; i++)
        a[i][0] += a[i-1][0];
    // max of left and up
    for (int i=1; i<m; i++) for (int j=1; j<n; j++)
        a[i][j] += max(a[i-1][j], a[i][j-1]);
    printf("%d\n", a[m-1][n-1]);
    return 0;
}
```

Longest Common Subsequence (LCS) 是序列分析的重要問題，一個序列的子序列是指將其中某些元素刪除後所得到的序列，字串可以看成字母組成的序列，以“algorithm”為例，“algtm”與“lgh”都是它的子序列，但是“agl”則不是，因為你不可以調整位置重新排列。輸入兩序列，LCS 要找一個最長的序列，它是兩輸入序列的共同子序列。LCS 可以是一種作為字串相似度的定義，有很多重要的應用。

P-6-7. LCS

輸入兩字串，計算其 LCS 的長度。

Time limit: 1 秒

輸入格式：第一行與第二行個有一個字串，字串均只含小寫字母，長度不超過 500。

輸出：LCS 長度。

範例一輸入： algorithm alignment	範例一輸出： 4
範例二輸入： fakefamefade acefaaaace	範例二輸出： 6

範例一說明：algm 或 algt 都是 LCS。

範例二說明：LCS 是 ae faae。

如果要暴力搜尋的話可慘了，一個長度 n 的序列有 2^n 個子序列，因為每一個位置都可以刪除或不刪除。假設第一個字串是 s_1 ，第二個是 s_2 ，以 DP 的思考我們來設法定義子問題，比對兩個字串最普通的方式是從前往後比對，我們定義 $lcs[i][j]$ 為： s_1 的前 i 個字元與 s_2 的前 j 個字元的 LCS 長度，也就是 s_1 與 s_2 的長度分別為 i 與 j 的 prefix 的 LCS 長度。我們分兩個情形：

- 如果 $s_1[i] \neq s_2[j]$ ：既然兩者不相等，兩者中至少有一個對找 LCS 是沒有幫助的，也就是說，我可以刪掉 s_1 的最後一個字母 $s_1[i]$ 或者刪掉 $s_2[j]$ ，因為要找越長越好的，所以兩者取其大，
 $lcs[i][j] = \max(lcs[i-1][j], lcs[i][j-1])$ 。

當然也可能兩者都刪除，因為是一直遞迴上去，兩者都刪的狀態是被包含在刪除其中一個的狀態之中。

- 如果 $s1[i]=s2[j]$ ：因為我們只計算 LCS 的長度，跟在哪個位置對中無關，所以既然對中，不對白不對，把其中一個刪除不會更好，因此可以得到

$$lcs[i][j]=lcs[i-1][j-1]+1。$$

我們已經完成遞迴式的設計了，邊界條件(初始條件)呢？依照我們定義的 $lcs[i][0]$ 與 $lcs[0][j]$ 代表其中一個字串的長度為 0 的狀況，長度為 0 字串與他人的 LCS 當然是空字串，所以 $lcs[i][0]=lcs[0][j]=0$ 。整理一下結果：

$$lcs[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ lcs[i-1][j-1] + 1 & \text{if } s1[i] = s2[j] \\ \max(lcs[i-1][j], lcs[i][j-1]) & \text{otherwise} \end{cases}$$

是不是有點眼熟呢？把 $lcs[][]$ 看成一個表格，每一個格子跟左方、上方與左上方的格子有關，當然還要看 $s1[i]$ 與 $s2[j]$ 是否相等。跟前一題一樣的計算順序就可以滿足 DP 避免遞迴的要求，所以由上而下，由左至右就是一個好的順序。以下是範例程式，時間複雜度是 $O(mn)$ ，因為每格只需要 $O(1)$ 。

```
// LCS, O(mn) space
#include <bits/stdc++.h>
using namespace std;
#define N 505
int main() {
    char s1[N], s2[N];
    int lcs[N][N];
    scanf("%s%s", s1, s2);
    int n=strlen(s1), m=strlen(s2), i, j;
    for (j=0; j<=n; j++) lcs[0][j]=0;
    for (int i=1; i<=m; i++) {
        lcs[i][0]=0;
        for (int j=1; j<=n; j++)
            if (s2[i-1]==s1[j-1])
                lcs[i][j] = lcs[i-1][j-1]+1;
            else
                lcs[i][j]=max(lcs[i-1][j], lcs[i][j-1]);
    }
    printf("%d\n", lcs[m][n]);
    return 0;
}
```

其實很多 DP 的程式都很簡單，遞迴式找出來之後，迴圈與表格從小到大拉過去就結束了。這一題我們多講一個空間上的問題，二維的問題有的時候會有空間用量太大的麻煩。如果兩個字串長度到 1 萬，時間或許還可以接受，但是空間會出現不足的問題。如果真

的需要這麼大的表格，那也是無可奈何，但如果只是要算 LCS 長度，根據上面的遞迴式，我們事實上並不需要一直留著整張表格。計算 $lcs[i][j]$ 時，我們只需要第 i 列與前一列就足夠了，所以有一個直接節省記憶體的方法：宣告一個 $2 \times N$ 的表格，永遠把第 0 列當作前一列，第 1 列當作第 i 列，也就是從第 0 列算到第 1 列，整列算完後將第 1 列複製到第 0 列，然後重複這個步驟。但是這麼做有個缺點就是多了複製的時間，雖然複雜度沒變。改善的方式是用輪流的：奇數回合從第 0 列算到第 1 列，偶數回合從第 1 列算回第 0 列。這個方式有人稱為滾動陣列，寫的方式不只一種，以下是其中一種寫法，交換變數 $from$ 與 to 我們用了 $swap()$ 函數，否則也可以用大家熟悉的交換方式。

```
// LCS, linear space
#include <bits/stdc++.h>
using namespace std;
#define N 505
int main() {
    char s1[N], s2[N];
    scanf("%s%s", s1, s2);
    int n=strlen(s1), m=strlen(s2);
    int lcs[2][N]={0}, from=0, to=1;
    for (int i=0; i<m; i++) {
        for (int j=1; j<=n; j++)
            lcs[to][j] = (s2[i]==s1[j-1])? lcs[from][j-1]+1: \
                max(lcs[to][j-1], lcs[from][j]);
        swap(from, to); // int temp=from; from=to; to=temp;
    }
    printf("%d\n", lcs[from][n]); // final result at [from]
    return 0;
}
```

Alignment 是生物序列 (如 DNA 與蛋白質) 分析的重要問題，輸入兩個字元序列，我們要將兩序列的字母依照原順序做一個對應，過程中可以將任一序列的某些位置加入空白，以輸入 "CTTAAC" 與 "CGGATCAT" 為例，以下是一個可能的 alignment (以 - 表示空白)：

```
CTTAAC-T
CGGATCAT
```

以下也是一個 alignment：

```
C---TTAACT
CGGATCA--T
```

alignment 的目的是要評估兩序列有多相似，我們會有一個評分機制，以下是一個例子：兩字母相同得 8 分，相異 -5 分，字母與空白對應 -3 分。給定評分機制，輸入兩序列，要計算最大的 alignment 分數。Alignment 可分為 global alignment 與

local alignment，前者是輸入的序列整個要納入 alignment，而後者是兩序列各選取連續的一段來做 alignment。

Q-6-8. Local alignment

輸入兩字串，計算 local alignment 的最大分數。評分機制為：兩字母相同得 8 分，相異-5 分，字母與空白對應-3 分。

Time limit: 1 秒

輸入格式：第一行與第二行各有一個字串，字串均只由 ATCG 四個字母組成，長度不超過 500。

輸出：Local alignment 的最大分數。

範例一輸入： ATATCTTAACTGG CGCGGATCATAA	範例一輸出： 43
範例二輸入： AAAAGTGAGGG GGCTATT	範例二輸出： 21

範例一說明：第一個字串取 ATCTTAA，第二個取 ATCATAA。

範例二說明：第一個字串取 CTGA，第二個取 CTA。

提示：計算 global alignment 與 LCS 很類似，計算 local alignment 與 global 的差別在於：可以“昨日種種譬如昨日死”，DP 過程中，如果前面的分數不好，可以放棄繼承，此外，最大分數未必出現在最後的位置。

下面一個例題是經典名題 0/1-knapsack，經典的原因是他在很多地方都是一個具有代表性的問題。

P-6-9. 大賣場免費大搬家

你抽中了大賣場的周年慶的抽獎活動，在不超過總重量 w 的限制下，你可任意挑選商品免費帶走。現場一共 n 項商品，每項商品有它的重量與價值，每項商品只可以選或不選，不可以拆開只拿一部份。請計算可以獲得的最大價值總和。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 w 。第二行有 n 個正整數，依序代表商品的重量，第三行有 n 個正整數，依序代表對應 n 項商品的價值。同一行數字間以空白隔開。 $n \leq 100$ ， w 與各商品重量及價值皆不超過 $1e5$ 。

輸出：最大價值總和。

範例一輸入： <pre>7 10 3 4 2 3 3 6 5 5 5 2 4 4 5 6</pre>	範例一輸出： <pre>14</pre>
範例二輸入： <pre>5 14 1 2 3 4 5 3 2 4 4 4</pre>	範例二輸出： <pre>15</pre>

範例一說明：選擇第 1, 2, 4 三項，重量為 $3+4+3=10$ ，價值為 $5+5+4=14$ 。

範例二說明：選擇第 1, 3, 4, 5 四項，重量為 $1+3+4+5=13$ ，價值為 $3+4+4+4=15$ 。

假設每個物品的價值與重量恰好成正比，或是更簡單的說，價值等於重量，這個問題就變成在總重量 w 限制下要裝越重越好，這個簡化版的問題稱為 Subset-sum 就是我們下一個習題。其實他更早就出現在我們第一章的 Q-1-8，當時是用來介紹 $O(2^n)$ 的暴搜法，物品數量小於 26。這個題目在沒有其他條件下，並沒有人發明出真正好的演算法，這裡說好的意思是時間複雜度為多項式而不會跑到指數函數。這一題我們把 n 的範圍放到 100，但增加了一個限制條件： w 的範圍不超過 $1e5$ ，也就是說在 w 不大的狀況下，有一個好的算法，這是這一題的經典之一，也是放在此處的主要目的。

以 DP 的思維模式，假設我們將子問題定義成 $dp[i]$ 是考慮前 i 個物品的最佳解，我們在設法列出遞迴式時會遭遇到困難，主要因素是加入一個新的物品時，前面找出來的最佳解可能全部改變，例如考慮前 5 項時，最佳解可能是挑第 1 與第 2 件，但是第六項納入考慮時，可能最佳解變成第 6 與第 3, 4 項。這樣找不到子問題最佳解之間的關係，所以是個失敗的定義。前面我們說過一個原則，找不出的時候常用的手法就是加條件，也可以說把解進一步分類。

我們考慮加入第二個條件：重量。令 $d[i][j]$ 是考慮前 i 項物品且重量不超過 j 的最佳解(最大可以獲得價值)，那麼遞迴式就好列了。假設第 i 項物品的重量是 $w[i]$ ，而價值是 $p[i]$ ，我們分成以下情形：

- $w[i] > j$ ：也就是根本不可能挑選，所以第 i 項物品有沒有都一樣， $d[i][j] = d[i-1][j]$ 。
- 第 i 項物品的重量不超過 j ，但是選擇不放：跟前一個情形一樣。
- 挑選第 i 項物品：既然已經挑選了第 i 項，那麼前 $i-1$ 項中挑選的重量不超過 $(j - w[i])$ ，因此， $d[i][j] = p[i] + d[i-1][j - w[i]]$ 。

最後的答案出現在 $d[n][W]$ ，而邊界條件為：沒有物品或重量限制 0 的時候的最大價值都是 0。整理一下結果：

$$d[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ d[i-1][j] & \text{if } w[i] > j \\ \max(d[i-1][j], p[i] + d[i-1][j - w[i]]) & \text{otherwise} \end{cases}$$

還是有點眼熟，把 $d[][]$ 看成一個表格，每一個格子跟上方與左上方的某個格子有關。所以我們依然可以用 i 由小到大， j 由小到大的順序來計算，以下是範例程式，時間複雜度是 $O(nW)$ ，其中 W 是總重限制，它跟之前看到的複雜度只有 n 不一樣，當 W 很大的時候 (例如 $1e20$)，這個程式很糟；但如果 W 不大 (例如本題 $1e5$) 這個程式跑起來要比暴搜好太多了，這個時間複雜度被稱為“偽多項式時間複雜度” (pseudo-polynomial time complexity)。

```
// 01-knapsack, O(Wn) space
#include <bits/stdc++.h>
using namespace std;
#define N 101
#define W 100005
int d[N][W]={0}; // max price of first i items with <= w weight
int main() {
    int n, total;
    scanf("%d%d", &n, &total);
    int w[N], p[N]; // weight and price
    for (int i=1; i<=n; i++)
        scanf("%d", &w[i]);
    for (int i=1; i<=n; i++)
        scanf("%d", &p[i]);
    for (int i=1; i<=n; i++) {
        for (int j=0; j<w[i]; j++)
            d[i][j]=d[i-1][j]; // i-th item cannot be included
        for (int j=w[i]; j<=total; j++) { // choose or not choose
            d[i][j]=max(d[i-1][j-w[i]]+p[i], d[i-1][j]);
        }
    }
    printf("%d\n", d[n][total]);
    return 0;
}
```

以下的習題是背包問題的退化形式，解法類似，留做習題，這題當然可以用前面介紹過的滾動陣列來節省記憶體，其實它甚至可以只用一維陣列，有興趣的人可以試試看。

Q-6-10. 置物櫃出租 (APCS201810)

王老先生有一個置物櫃，共有 M 個相同大小的格子，置物櫃目前已經租給 n 個客戶，第 i 位客戶所租的大小為 $f(i)$ 個格子 ($1 \leq i \leq n$)。目前的承租量總和不超過 M ，但是現在王老先生自己需要使用 s 個格子的置物櫃，如果剩下的容量不夠，就必須退掉某些客戶的租約。假設每個客戶所租容量只能全退或全不退，而退租第 i 個客戶損失的利益與其所租容量 $f(i)$ 相同，請寫一支程式計算王老先生最小的損失利益，如果不須退租，則損失為 0。

Time limit: 1 秒

輸入格式：測試資料有兩行，第一行有三個正整數，依序是 n 、 M 與 s ，其中 $s \leq M$ ，第二行是 n 個正整數 $f(1)$ ， $f(2)$ ， $f(3)$ ， $\dots$ ， $f(n)$ ，同一行的數字間以空白隔開。 $1 \leq n \leq 100$ ， $M \leq 2e5$

輸出：輸出王老先生最小的損失利益。

範例一輸入： 3 10 6 4 4 1	範例一輸出： 5
範例二輸入： 5 20 14 8 2 7 2 1	範例二輸出： 15

範例一說明：總共容量 10，已出租 9，剩下 1，需要 6，最少退租 4+1=5。

範例二說明：總共容量 20，已出租 20，剩下 0，需要 14，最少退租 8+7=15。

6.2.3. 1D1D

1D1D 的 DP 是指有 $O(n)$ 個子問題 (狀態)，但每一個子問題的計算需要 $O(n)$ 個子問題的結果，也就是說一個狀態會牽涉 $O(n)$ 個前置狀態。如果直接的計算，時間複雜度就是 $O(n^2)$ ，不過很多 1D1D 問題的計算都會利用資料結構或問題的特性來降低複雜度，

也就是因為如此，1D1D 的 DP 出現在競賽或考試場合的題目通常比前面看到的 1D0D 或 2D0D 的題目來得難，有些題目甚至可以說非常難。以下我們挑選一些適合這份教材難度的典型題目。

我們先從簡單的開始，這一個例題我們在第一章說明遞迴時曾經拿出來舉例，這是一個非常有名的數，興趣的可以去網路搜尋一下，這裡我們只關心如何計算。

P-6-11. Catalan number

Catalan number 的遞迴式定義：

$$C_0 = 1 \text{ and } C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i} \text{ for } n \geq 1.$$

計算 C_n 除以 P 的餘數， $P=1e9+9$ 。

Time limit: 1 秒

輸入格式：輸入一個非負整數 n ， $n < 100$ 。

輸出： C_n 除以 P 的餘數。

範例一輸入： 3	範例一輸出： 5
範例二輸入： 20	範例二輸出： 564120366

題目中直接給了 Catalan number 的遞迴定義，要計算它的第 n 項，因為數字很大，所以要求輸出除以 P 的餘數。遞迴式中涉及的運算只有加法與乘法，因此只要每次運算完都取一次餘數就可以了，而 P 的平方可能超過 `int` 的範圍但不會超過 `long long` 的範圍，所以計算過程要使用 `long long`，但要記得每一次運算完都要取餘數，不可以幾個運算寫在一起算完了才取餘數，那就有可能在過程中溢位。雖然這一題正常的 DP 寫法就很容易，以下我們用它來示範 top-down 的寫法。

以下是範例程式。Top-down DP 的重點是準備一個 `memo[]` 來記錄算過的答案，主程式第 19 行我們將 `memo` 初設為 -1 來表示沒算過，接著依照定義 `memo[0]=1`，然後就呼叫遞迴函數 `cat(n)` 計算與輸出答案。函數 `cat(n)` 一進入時先檢查 `memo[n]` 是否大於等於 0，若是，表示曾經算過，直接回傳答案；否則，依照定義在第 8~12 行利用遞

迴計算，最後在第 13 行回傳，且回傳前先存入 `memo[n]` 中。如果不使用 `memo` 的技術，純遞迴會非常耗時間，但如果這樣寫，時間複雜度跟 `bottom-up` 的一樣都是 $O(n^2)$ 。

```

00 // P-6-11 Catalan number // top-down
01 #include <bits/stdc++.h>
02 using namespace std;
03 typedef long long ll;
04 #define P 1000000009
05 ll memo[101]={0};
06 ll cat(int n) {
07     if (memo[n]>=0) return memo[n];
08     ll sum=0;
09     for (int i=0; i<n;i++) {
10         ll t=cat(i)*cat(n-1-i)%P;
11         sum = (sum+t)%P;
12     }
13     return memo[n]=sum;
14 }
15
16 int main() {
17     int n;
18     scanf("%d",&n);
19     for (int i=0;i<=n;i++)
20         memo[i]=-1;
21     memo[0]=1;
22     printf("%lld\n",cat(n));
23     return 0;
24 }

```

所附檔案中還有兩支範例程式，一支是正常 `bottom-up` 的寫法，另外一支是運用數學解，Catalan number 在數學上可以解出來，第 n 項的值是 $C(2n, n) / (n+1)$ ，其中 $C(2n, n)$ 是 $2n$ 個相異物取 n 個的組合數，所以我們可以直接用公式計算它的值，但是因為中間涉及除法，而模 P 的運算不可以直接使用除法，需要轉換成乘以除數的模反元素。計算 Catalan number 是解遞迴關係(差分方程)一個經典例子，但這些都超過中學生的數學範圍，在離散數學的課程中可以學到。上面這段補充說明只是給有興趣的人參考，如果覺得困難，你可以略過。

下面這一題是 P-6-2 的推廣，留做習題。

Q-6-12. 楊鐵心做 1 休 K

楊鐵心帶著義女穆念慈做武術表演已經很久了，因為有些表演(例如胸口碎大石)實在很傷身體，所以現在每次表演完都要休息養傷。他接到許多的邀約，每天均有一場。每一場表演都可以得到某些金額的報酬，但是每次表演完至少要休息 K 天，請你協助他決定應該接受那些表演以得到最大的報酬。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 K 。第二行有 n 個正整數，依序代表第 1 天開始每天邀約報酬，數字間以空白隔開。 $K < n \leq 1e5$ ，每天酬勞不超過 10000。

輸出：最大可能獲得的總酬勞。

範例一輸入： 5 1 1 2 3 1 5	範例一輸出： 9
範例二輸入： 8 3 2 1 1 7 3 2 9 2	範例二輸出： 11

範例一說明：挑選 $1+3+5=9$ 。

範例二說明：挑選 $2+9=11$ 。請注意，如果挑了 9，就不可以挑 7，兩者只隔 2 天。

下一題是 P-6-3 的推廣型。

P-6-13. 周伯通的基地台 (@@)

周伯通開了一家通訊大廠並以他的名字命名，就叫做伯通公司，也有不少人弄錯了稱為博通。周伯通標下了一個政府標案，要在一條長長的大街架設基地台，長街被劃分成 n 個區段，編號依序為 $1 \sim n$ 。在第 i 個區段架設基地台的話，需要成本 $c[i]$ ，而可以服務第 $i-k$ 到第 $i+k$ 的區段 (超出範圍可忽略)。現在輸入 k ，要選一些區段架設基地台，以便每一個區段都可以被服務到，請計算最少的總成本。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 k 。第二行有 n 個正整數，依序代表 $c[1]$, $c[2]$, ..., $c[n]$ ，數字間以空白隔開。 $k < n \leq 2e5$ ，每處成本不超過 1000。

輸出：最小總成本。

範例一輸入： 5 1 1 2 3 1 5	範例一輸出： 2
--------------------------------	-----------------

範例二輸入： 8 2 2 1 1 7 3 2 9 2	範例二輸出： 3
範例一說明：挑選 $1+1=2$ 。 範例二說明：挑選 $1+2=3$ 。	

若 $k=1$ ，這一題就是 P-6-3。令 $p(i)$ 表示可以服務 i 之前的所有位置而最後一個站架在 i 最小成本，因為與前一站的距離不可超過 $2k+1$ ，所以前一站必然在 $[i-2k-1, i-1]$ ，我們可以得到

$$p(i) = c[i] + \min\{p(j) : i-2k-2 < j < i\} \text{ for } i > k+1; \text{ and } \\ p(i) = c[i] \text{ for } i \leq k+1$$

這裡假設超出範圍的 $p(i)$ 為無窮大。最後的解在 $\min\{p(i) : n-k \leq i \leq n\}$ 。如果直接做，需要 $O(kn)$ 的時間，所以我們要設法很快的求得區間最小值，有一些比較高階的資料結構可以每次求的區間最小值 (RMQ, range minimum query)，但那超出了這份講義目前要講的範圍。在 P-3-8 例題中，我們曾經處理過這樣的問題：在一個滑動的視窗中每次可以找出最小值，所以這一題要綜合之前學過的這個技術，算是比較困難的題目。

回顧 P-3-8 的做法，基本上我們啟用一個 deque (雙向佇列)，這一題只會往前滑動，所以自製也容易。在 deque 中我們只須要維護一個遞增序列，因為後面如果有碰到比較小的，前面的就沒用了。所以 deque 的前端就是區間最小值，每回合先檢查前端是否已出界，然後把新的從後端放進 deque 並且移除任何大於它的資料。以下是盡量不使用 STL 的寫法，如果還沒有裝備 STL 的人可以使用這樣的寫法。

```
// P-6-13 min cost of k-domination O(n), no STL
// p[i] = min cost to cover [1..i] and choosing i
// p[i] = c[i] + min{p[j] : j > i-2k-2}; and p[i] = c[i] for i <= k+1
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define N 200005
int main() {
    int n, k, c[N], p[N];
    scanf("%d%d", &n, &k);
    for (int i=1; i<=n; i++)
        scanf("%d", &c[i]);
    // deque, index of p[], increasing p[]
    int minq[N], head=0, tail=0;
```

```

p[1]=c[1];
minq[tail++]=1; // push_back
for (int i=2; i<=n; i++) {
    if (i<=k+1)
        p[i]=c[i];
    else {
        if (minq[head]<i-2*k-1)
            head++; // out of range, pop_front
        p[i] = p[minq[head]] + c[i];
    }
    // pop larger and push itself
    while (head<tail && p[minq[tail-1]]>=p[i])
        tail--;
    minq[tail++]=i; // push i
}
// find answer: min p[n-k:n]
int ans=p[n-k];
for (int i=n-k+1; i<=n; i++)
    if (p[i]<ans) ans=p[i];
printf("%d\n", ans);
return 0;
}

```

以下是使用 STL 的 deque 的寫法。

```

// P-6-13 min cost of k-domination
// p[i]= min cost to cover [1..i] and choosing i
// p[i] = c[i]+min{p[j]: j>i-2k-2}; and p[i]=c[i] for i<=k+1
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define N 200005
int main() {
    int n,k;
    int c[N];
    scanf("%d%d", &n, &k);
    for (int i=1; i<=n; i++)
        scanf("%d", &c[i]);
    int p[N];
    deque<int> minq; // index of p[], increasing
    p[1]=c[1];
    minq.push_back(1);
    for (int i=2; i<=n; i++) {
        if (i<=k+1)
            p[i]=c[i];
        else {
            if (minq.front()<i-2*k-1)
                minq.pop_front();
            p[i] = p[minq.front()] + c[i];
        }
        // pop larger and push itself
    }
}

```

```

        while (!minq.empty() && p[minq.back()]>=p[i])
            minq.pop_back();
        minq.push_back(i);
    }
    printf("%d\n", *min_element(p+n-k, p+n+1));
    return 0;
}

```

因為我們之前也教了 PQ (priority_queue)，這一題因為區間一直往右滑動，所以區間的最小值也可以用 PQ 來做，不過前兩個寫法的複雜度只有 $O(n)$ ，而使用 PQ 是 $O(n \log(n))$ ，而且這個寫法基本上是一種偷懶的方法，如果不是走競賽程式的人，不推薦學這種方式。

```

// P-6-13 min cost of k-domination, using PQ
// p[i] = min cost to cover [1..i] and choosing i
// p[i] = c[i] + min{p[j]: j > i-2k-2}; and p[i] = c[i] for i <= k+1
#include <bits/stdc++.h>
using namespace std;
#define N 200005
int main() {
    int n, k, c[N], p[N];
    scanf("%d%d", &n, &k);
    for (int i=1; i<=n; i++)
        scanf("%d", &c[i]);
    priority_queue<pair<int,int>> pq; // (-p[i], i)
    p[1]=c[1];
    pq.push({-p[1], 1});
    for (int i=2; i<=n; i++) {
        if (i<=k+1)
            p[i]=c[i];
        else {
            // remove out-of-range minimum
            while (pq.top().second<i-2*k-1)
                pq.pop();
            p[i] = -pq.top().first + c[i];
        }
        pq.push({-p[i], i});
    }
    printf("%d\n", *min_element(p+n-k, p+n+1));
    return 0;
}

```

下一題是 P-4-12. (一次買賣) 的推廣型，

Q-6-14. K 次買賣 (106 高中全國賽 subtask)

某商品在某個時期每一天的價格是 $p[1], p[2], \dots, p[n]$ 。假設只能先買後賣，且賣了之後才能再買，請計算買賣 K 次的最大獲利總價差，允許當天買賣，也就是不超過 K 次的買賣。

Time limit: 1 秒

輸入格式：第一行是正整數 n 與 K ，第二行有 n 個正整數 $p[1], p[2], \dots, p[n]$ 。
 n 不超過 $1e5$ ， $K < 100$ ，商品價格皆不超過 $1e7$ 。

輸出：買賣不超過 K 次的最大總價差。

範例一輸入： <pre>5 1 3 5 1 4 0</pre>	範例一輸出： <pre>3</pre>
範例二輸入： <pre>7 2 1 3 7 5 1 4 0</pre>	範例二輸出： <pre>9</pre>

範例一說明：買 1 賣 4。

範例二說明：買 1 賣 7，再買 1 賣 4。

(本題有效率更高的解法，但難度太高，這裡只要求 $O(kn)$ 的解)

Q-6-14 提示：令 $f(i, j) =$ 在第 j 天之內完成不超過 i 次買賣的最大獲利，則
 $g(i, j) = \max\{p[j] - p[t] + f(i-1, t-1) : \text{for all } t < j\}$ 是在第 j 天最後賣出的最大獲利，顯然算出 g 之後再取一次的 prefix maximum 就可以得到 f 。運用一個 i 迴圈跑 k 次，在 i -迴圈內，對每一個 j 點找出對應的 t 就可以算出 $g()$ 與 $f()$ 了。

P-6-15. 一覽衆山小

所謂「會當凌絕頂，一覽衆山小。」很多人都想爬到高處，為的是將群山看小，但是登高必自卑，行遠必自邇，最好是一步一步逐步往上，妄想一步登天的人往往摔的慘重。小說與遊戲中出現的人物往往都是越晚出現越厲害，現在已知所有人物的戰力與出場順序，想要找出依照出場順序而且戰力逐步上升的人物序列，請你計算滿足這樣要求的序列的最大可能長度。

Time limit: 1 秒

輸入格式：第一行有一個正整數 n 代表出場的人物數。第二行有 n 個非負整數，是依出場順序的每一位人物的戰力，數字間以空白隔開。 $n \leq 2e5$ ，戰力值不超過 $1e8$ 。

輸出：出場順序與戰力皆遞增的序列的最大可能長度。

範例一輸入： 8 2 4 1 3 6 4 6 9	範例一輸出： 5
範例二輸入： 5 5 4 3 2 1	範例二輸出： 1

範例一說明：可以挑選子序列 (1, 3, 4, 6, 9)，長度為 5。

範例二說明：輸入是個遞減序列，找不到長度大於 1 的遞增子序列。

一個序列的子序列是指可以任意挑選某些成員，但是不可以交換他們的前後順序，也可以說是刪除某些元素。本題是要在一個數列中找到一個最長的遞增子序列，所以這個問題被稱為 LIS (longest Increasing subsequence)。這算是一個經典的 1D1D 的 DP，因為不會太困難但是又需要一些技巧。

假設序列是 $S[1:n]$ ，從 DP 的思維開始想，我們定義 $lis(i)$ 是以第 i 點結束的最長 LIS 長度，因為前一點一定要比 $S[i]$ 小，所以很容易可以寫出遞迴式，為了方便起見，我們在序列前方加一個 -1，這樣便不需要初始條件了：

$$lis(i) = \max\{lis(j) + 1 : j < i \text{ and } S[j] < S[i]\}。$$

如果直接實作這個遞迴式，顯然需要 $O(n^2)$ ，因為每個 i 要往前搜尋所有它前面的點。考慮有那些沒有用的計算，假設我們已經做到第 i 個點。對於 $j < i$ ，若 $S[i] \leq S[j]$ 而 $lis(i) \geq lis(j)$ ，也就是 $S[i]$ 是比較小的結尾但是有比較長的 LIS，那麼 j 顯然是沒有用的，因為可以接在 $S[j]$ 後面的一定也可以接在 $S[i]$ 後面，而接在 $S[i]$ 後面可以得比較好的結果。如果我們刪除那些沒有用的子問題結果，會剩下甚麼呢？每一種可能的長度只會有一個點被留下來， lis 長度為 L 而被留下來的就是「長度 L 的最小可能結尾」。如果我們將這個資料存放在一個陣列 $last[]$ ，這個陣列的元素是單調遞增的，在第 i 個回合計算 $lis(i)$ 時，我們搜尋找到 $L = \max(j : last[j] < S[i])$ ，於是我們得到 $lis(i) = L + 1$ ，最後我們要將 $lis(i)$ 的資料納入，也就是說 $L + 1$ 的長度

多一個可能結尾是 $S[i]$ ，因此 $last[L+1]=\min(last[L+1], S[i])$ 。請稍等，還記得 L 是怎麼來的嗎？ L 是小於 $S[i]$ 的最大的那一個 $last[]$ ，也就是說除非 L 是 $last[]$ 的最後一個元素，否則 $last[L+1] \geq S[i]$ ，因此根本不必多做一次比較。

以下的範例程式是直接根據這樣的思考寫的，作法很直接，序列也不必先讀進來存，讀一個做一個就可以。雖然比 $O(n^2)$ 好，但是還是不太好，因為時間複雜度其實是 $O(nL)$ ，其中 L 是 LIS 的長度，也就是 $last[]$ 的長度，在最壞的狀況還是 $O(n^2)$ 。

```
// P-6-15 LIS  $O(nL)$ ,  $L$  is the length, linear search
#include <bits/stdc++.h>
using namespace std;
#define N 200010

int main() {
    int n;
    scanf("%d", &n);
    int last[N], len=0; // last[i]=the min end of length i+1
    // last will be increasing
    for (int i=0; i<n; i++) {
        int si;
        scanf("%d", &si);
        int r = 0;
        // find the max last[r] < t
        while (r<len && last[r]<si)
            r++;
        last[r] = si;
        if (r==len) len++;
    }
    printf("%d\n", len);
    return 0;
}
```

記得 $last[]$ 是個單調遞增的陣列嗎？長度越長的結尾越大，既然是單調陣列當然就可以二分搜了，二分搜可以用 STL 的 `lower_bound()`，如果還不熟也可以自己寫，建議用我們第二章介紹的一路往前跳的方式而避免使用左右轉頭搜的方式。以下的範例程式中，我們把兩種方法寫在同一個程式中以供參考，時間複雜度都是 $O(n \log(L))$ 。

```
// P-6-15 LIS  $O(n \log(L))$ ,  $L$  is the length
#include<bits/stdc++.h>
using namespace std;
#define N 200010
// binary search to find the first  $\geq x$ 
int bsearch(int s[], int n, int x) {
    if (n==0 || s[0]>=x) return 0;
    int po=0;
    for (int jump=n/2; jump>0; jump>>=1) {
```

```

        while (po+jump<n && s[po+jump]<x)
            po += jump;
    }
    return po+1;
}
int main() {
    int n;
    scanf("%d",&n);
    int last[N], len=0; // last[i]=the min end of length i+1
    // last will be increasing
    for (int i=0;i<n;i++) {
        int si;
        scanf("%d",&si);
        // one of the next two binary search
        int r = bsearch(last, len, si);
        // only for checking
        int it = lower_bound(last, last+len, si) - last;
        assert(r==it);
        last[r] = si;
        if (r==len) len++;
    }
    printf("%d\n",len);
    return 0;
}

```

下面在附上用 vector 的寫法，背後有不簡單的思維，但最後結果非常簡潔，無怪乎成為經典。

```

// P-6-15 LIS, using STL
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, si;
    scanf("%d",&n);
    vector<int> last;
    for (int i=0; i<n; i++) {
        scanf("%d", &si);
        auto it = lower_bound(last.begin(), last.end(), si);
        if (it==last.end()) last.push_back(si);
        else *it=si;
    }
    printf("%d\n", (int)last.size());
    return 0;
}

```

以下也是個經典例題，它是 P-4-4 的加權版本。

P-6-16. 山寨華山論劍

自從華山論劍聲名大噪之後，很多人都想參加，但 20 年辦一次實在太久了，五輪奧運都跑 5 趟了，於是各地都紛紛舉辦華山論劍，往往同一天就有很多場。每一場都有自己的開始時間與結束時間，參加者必須全程在場，所以兩場的時間如果有重疊就不能同時參加。令狐冲本來想參加最多場來盡速累積經驗值，但後來他發現有很多場次能獲得的經驗值很少，參加了反而浪費時間，現在要請你幫忙計算最多可以得到的經驗值總和。請注意，所挑選活動必須完全不重疊，兩場活動只在端點重疊也是不可以同時參加的，也就是說前一場的結束時間必須小於下一場的開始時間。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，代表一共舉辦了 n 場華山論劍，接下來 n 行，每行有三個非負整數 s 、 t 與 e ，代表這場活動時間區間是 $[s, t]$ ，而可以得到的經驗值是 e ，兩數字間以空白間格。 n 不超過 $1e5$ ， $s < t < 1e9$ ，答案不超過 $1e9$ 。

輸出：最大可能經驗值總和。

範例一輸入： <pre>3 1 2 1 2 3 3 3 4 1</pre>	範例一輸出： <pre>3</pre>
範例二輸入： <pre>3 1 2 2 2 3 3 3 4 2</pre>	範例二輸出： <pre>4</pre>

範例一說明：挑選 $[2, 3]$ ，得到經驗值 3。

範例二說明：挑選 $[1, 2]$ 與 $[3, 4]$ ，得到經驗值 $2+2=4$ 。

每一場華山論劍(活動)看成一個線段，每個線段除了有起點(左端)與終點(右端)之外，還有一個權重。問題是要挑選權重總和最大且互相不重疊的線段，這個問題也稱為 weighted activity selection problem，如果權重都一樣，就是要挑選最多活動，那就是第四章的 P-4-4，沒權重的時候是用貪心法則來求解，挑選最早結束的活動

就對了，但是在加權版本時，範例一告訴你，挑最早結束的是錯的，範例二告訴你，挑權重最大的也是錯的。

放棄貪心來試試 DP。假設線段放在 $s[]$ 中，第 i 個線段的區間是 $[s[i].L, s[i].R]$ ，權重是 $s[i].w$ 。先讓線段依照右端排序，定義 $dp[i]$ 是前 i 個線段的最佳解 (最大權重和)。第 i 個線段可能挑或不挑，不挑的話，很簡單 $dp[i]=dp[i-1]$ ；如果挑呢？跟它重疊的都不能挑，也就是說右端落在 $s[i]$ 範圍內的線段都要剔除，根據 $dp[i]$ 的定義，我們暫且不需要管 $i+1$ 以後的線段。剔除了右端落在 $s[i]$ 範圍的線段就是右端在 $s[i].L$ 之前的線段，也就是 $\max\{dp[j]: s[j].R < s[i].L\}$ 。因為 $dp[]$ 顯然是非下降的，所以我們要找的 $dp[j]$ 就是滿足右端小於 $s[i].L$ 的最大編號 j 。整理一下我們的遞迴式，選或不選 i ，兩者挑最大：

$$dp[i] = \max\{dp[i-1], s[i].w + dp[j]\}, \quad j = \max\{x: s[x].R < s[i].L\}$$

從狀態轉移的角度來看， i 的前置狀態有兩個，一個是 $i-1$ (不選 i)，另外一個就是在它開始之前最後一個結束的線段 (選 i)，這樣看其實不難理解，當然，要找到 j 需要一點技術。先提出根據上面的分析後的天真作法，排序後一一尋找，雖然說天真但也不太單純，我們需要排序一個三個欄位的資料，因此定義了一個 struct 來放置線段，定義了一個比較函數以便依照右端排序，不熟悉的話可能需要複習第二章。排序後以一個 i 迴圈每次計算 $dp[i]$ ，其中以一個 j 迴圈來找到 i 開始前的最後一個線段，如果沒有，之前的最大權重相當於 0，程式很直接，複雜度 $O(n^2)$ 。

```
// Weighted activity selection problem, O(n^2) method
// overlap at endpoint is not allowed
// dp[i] is the max for interval until right(i), sorted by right
// dp[i]=max{dp[i-1], dp[j]+w[i]}, j is the max right < left(i)
#include<bits/stdc++.h>
using namespace std;
#define N 200010
typedef struct {
    int L,R,W;
} SEG;
SEG seg[N];
// sort by right, same right points OK since dp[] is increasing
bool comp(const SEG p1, const SEG p2) {
    return p1.R < p2.R;
}

int main() {
    int n, dp[N];
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d%d%d", &seg[i].L, &seg[i].R, &seg[i].W);
    sort(seg, seg+n, comp); //sort by right
    dp[0]=seg[0].W;
    for (int i=1; i<n; i++) {
```

```

    // linear search largest j< left[i]
    int j=i-1;
    while (j>=0 && seg[j].R>=seg[i].L)
        j--;
    dp[i] = seg[i].W + ((j<0)? 0 : dp[j]);
    if (dp[i]<dp[i-1]) dp[i]=dp[i-1];
}
printf("%d\n",dp[n-1]);
return 0;
}

```

因為右端是經過排序的，當然應該用二分搜比較有效率。為了簡化邊界條件，我們可以塞一個右端為-1 而權重為 0 的線段，如此一來每一個每一個線段都找得到它的前置線段。以下是自己寫二分搜的範例程式。

```

// Weighted activity selection problem, using binary search.
// overlap at endpoint is not allowed
// dp[i] is the max for interval until right(i), sorted by right
// dp[i]=max{dp[i-1], dp[j]+w[i]}, j is the max right < left(i)
#include <bits/stdc++.h>
using namespace std;
#define N 200010
typedef struct {
    int L,R,W;
} SEG;
SEG s[N];
// sort by right, same right points OK since dp[] is increasing
bool comp(const SEG p1, const SEG p2) {
    return p1.R < p2.R;
}

int main() {
    int n, dp[N];
    scanf("%d", &n);
    for (int i=1; i<=n; i++) {
        scanf("%d%d%d", &s[i].L, &s[i].R, &s[i].W);
        assert(s[i].L<s[i].R);
    }
    s[0].R = -1; // dummy for boundary
    sort(s, s+n+1, comp); //sort by right
    dp[0] = 0;
    for (int i=1; i<=n; i++) {
        // binary search to find largest j< left[i]
        int j=0;
        for (int jump=i/2; jump>0; jump>>=1) {
            while (j+jump<i && s[j+jump].R<s[i].L)
                j += jump;
        }
        dp[i] = dp[j] + s[i].W;
        if (dp[i]<dp[i-1]) dp[i] = dp[i-1];
    }
}

```

```
    }
    printf("%d\n", dp[n]);
    return 0;
}
```

當然二分搜也可以呼叫 `lower_bound()`，但是要注意比較函數的寫法，另外也可以用 `map` 來做，範例程式中有這兩種寫法，提供有興趣的人參考，這裡就不多說明了。

6.2.4. 2D1D 與其他

P-6-17. 切棍子

有一台切割棍子的機器，每次將一段棍子會送入此台機器時，我們可以選擇棍子上的切割位置，機器會將此棍子在指定位置切斷，而切割成本是該段棍子的長度。現在有一根長度 L 的棍子，上面標有 n 個需要切割的位置座標，因為不同的切割順序會影響切割總成本，請計算出最小的切割總成本。

例如 $L=10$ ，三個切割點的座標是 $(2, 4, 7)$ 。如果切割順序是 $(2, 4, 7)$ ，則第一次切的成本是 10，第二次的成本是 8，第三次成本 6，總成本 $10+8+6=24$ 。如果切割順序改成 $(4, 2, 7)$ ，第一次切的成本是 10，切成長度 4 與 6 的兩段，第二次的成本是 4，第三次成本 6，總成本 $10+4+6=20$ 。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 L 。第二行有 n 個正整數，依序代表由小到大的切割點座標 $p[1] \sim p[N]$ ，數字間以空白隔開，座標的標示的方式是以棍子左端為 0，而右端為 L 。 $N \leq 200, L \leq 1e6$ 。

輸出：最小的切割總成本。

範例一輸入： 3 10 2 4 7	範例一輸出： 20
範例二輸入： 5 10 1 6 7 8 9	範例二輸出： 24

從題目給的範例，很可能讓我們猜想每次應該取中點或 median 來切割，但事實上都是錯的。直接思考 DP 的做法，定義一段棍子的切割成本最直接的方式就是以左右端點，因為是有給座標的，我們就以所給的座標當作第 $1, 2, \dots, n$ 點，定義 $\text{cost}(i, j)$ 是第 i 點到第 j 點之間這段的最低切割成本，為了方便，以座標 0 與 L 當作第 0 與第 $n+1$ 點，所以我們要計算的就是 $\text{cost}(0, n+1)$ 。

對任意 $i < j$ ， $\text{cost}(i, j)$ 就是一個子問題，所以我們有 $O(n^2)$ 個子問題，現在要找出遞迴式。對於 $j=i+1$ ， $\text{cost}(i, j)=0$ ，因為中間沒有要切割的點；否則 $j>i+1$ ，假設第一刀切在 k ，我們雖然不知道 k 在哪裡，但是一旦第一刀切在 k 之後，剩下兩段的成本就是 $\text{cost}(i, k)$ 與 $\text{cost}(k, j)$ 。DP 的思維：既然不知道 k 是多少，那就全部都算吧！所以我們可以得到

$$\text{cost}(i, j) = \begin{cases} 0 & \text{if } j = i + 1 \\ \min_{i < k < j} \{ \text{cost}(i, k) + \text{cost}(k, j) \} + p[j] - p[i] & \text{otherwise} \end{cases}$$

其中 $p[j]$ 與 $p[i]$ 是兩點的座標。這是個遞迴式嗎？是的，雖然跟之前看到的有點不一樣，遞迴都是大往小，我們注意等號右邊的座標間格 $(k-i)$ 與 $(j-k)$ 都會小於左邊的 $(j-i)$ ，所以它是區間由大到小的遞迴，類似二分搜尋與排序的狀況。所以這一題的計算順序稍嫌麻煩，是一個試試 top-down 好時機。我們可以看到寫 top-down DP 有多麼簡單（在你找出遞迴式之後）。

以下是範例程式，基本上直接寫遞迴函式差不多。一開始把輸入座標讀進來，然後準備小抄 memo，因為我們的子問題是所有 (i, j) ，就宣告一個 `int memo[N][N]`，有一半用不到不需要太吝嗇，第 23~25 行將它們初設為 -1，用來代表沒有計算過，而將所有 `memo[i][i+1]` 設為 0，然後就直接呼叫 `cost(0, n+1)` 輸出。在函數 `cost(i, j)` 中，一進入時先檢查對應的 `memo[i][j]` 是否不是負的（第 9 行），若是，表示曾經算過，便直接回傳表格內容。否則我們要進行遞迴呼叫來計算，第 10~12 行就是根據遞迴式寫的計算各種第一刀 k 的可能，在其中取最小值，計算完畢後在第 14 行要回傳前先將他寫入表格 `memo[i][j]` 中。是不是很简单？這個程式的複雜度是多少呢？直接看遞迴不容易，看表格就好，有 $O(n^2)$ 個格子，每個格子一個不超過 $O(n)$ 的迴圈，所以就是 $O(n^3)$ 。這一題是存在優化的方法，不過這些優化需要依靠一些數學上的性質，不是我們這份教材的範圍。

```
00 // P-6-17 rod-cutting, top-down DP  $O(n^3)$ 
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 600
04 #define oo 99999999
05 int p[N]; // points
06 int memo[N][N];
07 // min cost of [i, j]
```

```

08 int cost(int i, int j) {
09     if (memo[i][j]>=0) return memo[i][j];
10     int mincost = oo;
11     for (int k=i+1; k<j; k++)
12         mincost = min( mincost, cost(i,k) + cost(k,j));
13     mincost += p[j] - p[i];
14     return memo[i][j]=mincost;
15 }
16
17 int main() {
18     int n,L;
19     scanf("%d%d", &n, &L);
20     for (int i=1; i<=n; i++)
21         scanf("%d",&p[i]);
22     p[0]=0, p[n+1]=L;
23     for (int i=0; i<n+2; i++)
24         for (int j=i+1; j<n+2; j++)
25             memo[i][j]=-1;
26     for (int i=0; i<n+1; i++)
27         memo[i][i+1] = 0;
28     printf("%d\n", cost(0,n+1));
29     return 0;
30 }

```

如果要寫非遞迴非 top-down 的寫法要如何寫呢？其實也不是多難，我們剛才提到子問題的大小看 $j-i$ ，所以非遞迴的順序要依照 $j-i$ 由小到大計算，如果看成一個矩陣，計算的順序是由對角線逐漸往右上角算。底下是非遞迴的範例程式，要小心迴圈的邊界，這個程式裡面是用一個變數代換 $len=j-i$ ，這樣比較好思考也比較不容易寫錯。

```

// P-6-17 rod-cutting, bottom-up DP  $O(n^3)$ 
#include <bits/stdc++.h>
using namespace std;
#define N 600
#define oo 999999999

int main() {
    int n, L, p[N], cost[N][N];
    scanf("%d%d", &n, &L);
    for (int i=1; i<=n; i++)
        scanf("%d",&p[i]);
    p[0]=0, p[n+1]=L;
    for (int i=0; i<n+1; i++)
        cost[i][i+1] = 0;
    // len=j-i
    for (int len=2; len<n+2; len++) {
        for (int i=0; i+len<n+2; i++) { // cost(i, i+len)
            int j=i+len, mcost = oo;
            for (int k=i+1; k<j; k++)
                mcost = min(mcost, cost[i][k]+cost[k][j]);
            cost[i][j] = mcost + p[j] - p[i];
        }
    }
}

```

```

    }
}
printf("%d\n", cost[0][n+1]);
return 0;
}

```

下面一題是類似的習題，也是 2D1D 的 DP，這裡同樣只要求 $O(n^3)$ 的複雜度，這一題同樣存在考數學性質的優化。

Q-6-18. 矩陣乘法鏈

兩個矩陣 A 與 B 要相乘，必須滿足 A 的行數等於 B 的列數，若 A 是 $p \times q$ 的矩陣，B 是 $q \times r$ 的矩陣，則 $A \times B$ 的結果是一個 $p \times r$ 的矩陣，而需要的純量乘法數假設是 $p \times q \times r$ 。矩陣乘法滿足結合律，也就是說 $A \times B \times C = (A \times B) \times C = A \times (B \times C)$ ，可是不同順序所需要的純量乘法數不同。輸入 $p[0], p[1], \dots, p[n]$ ，代表有 n 個矩陣相乘的乘法鏈，而它們的矩陣的大小依序分別是 $p[0] \times p[1]$ 、 $p[1] \times p[2]$ 、 $\dots$ 、 $p[n-1] \times p[n]$ ，請找出最好的相乘順序使得使用的純量乘法數的總和最少。

舉例來說， $n=3$ ，矩陣大小為， $A_1: 3 \times 5$ ， $A_2: 5 \times 4$ ， $A_3: 4 \times 2$

乘法順序為 $(A_1 \times A_2) \times A_3$ 時，乘法數 $3 \times 5 \times 4 + 3 \times 4 \times 2 = 84$ ；若乘法順序為 $A_1 \times (A_2 \times A_3)$ 時，乘法數 $3 \times 5 \times 2 + 5 \times 4 \times 2 = 70$ 。最少的純量乘法數為 70。

Time limit: 1 秒

輸入格式：第一行是正整數 n 。第二行有 $n+1$ 個正整數， $p[0] \sim p[n]$ ，數字間以空白隔開。 $n \leq 200$ ， $p[i] \leq 200$ 。

輸出：最小的純量乘法數量。

範例一輸入： 3 3 5 4 2	範例一輸出： 70
範例二輸入： 4 5 1 3 4 2	範例二輸出： 30

下一題出現在 APCS 的考試中，算是歷屆比較難的題目，因為不在常見的套路中。

P-6-19. 階梯數字 (APCS201802) (@@)

一個正整數如果從高位數往低位數看過去，每一位數字只會相等或變大，則我們稱它為階梯數字，例如：9、234、777、11222233。給定一正整數 N ，請計算不大於 N 的階梯數字總共有幾個，請注意本題只算正整數，所以 0 不算階梯數字，而且階梯數字不會以 0 開始。

提示：對於 N 不大的情形，例如 100000，我們可以枚舉每一個不超過 N 的數字來判斷。對於 N 很大的時候，就需要比較快速的方法了。我們知道所有的一位數 1~9 都是階梯數字，如果要計算 D 位數的階梯數字總數，我們可以依據最高位數字分成 9 類，然後根據 $(D - 1)$ 位的 9 類階梯數字總數來計算。

Time limit: 1 秒

輸入格式：輸入一個正數字 N 。 $N \leq 1e18$ 。

輸出：不大於 N 的階梯數字個數。

範例一輸入： 25	範例一輸出： 22
範例二輸入： 101	範例二輸出： 54

範例一說明：1~9，11~19，22~25，共 22 個。

範例二說明：1~9，11~19，22~29，33~39，44~49，55~59，66~69，77~79，88~89，99，一共有 54 個。

要判斷一個數字是否是階梯數字並不難，只要把每一位數字拆解出來再檢查是否單調上升就可以了，题目的提示中也提到，數字範圍小的時候可以枚舉，但 N 很大時就必須想快速的方法了。這一題比較需要思考，我們先來看看如何以枚舉來解簡單的子題。以下是範例程式，我們設計一個副程式 `step(n)` 用來判斷 n 是否是階梯數字，將 n 從低位往高位拆解，如果有上升的狀況即回傳 0，如果都沒有，最後回傳 1。主程式中就把 1 開始的所有數字都拿來檢查看看。

```
// P_6-19 slow method
#include <cstdio>
typedef long long LL;
```

```

int step(LL n) {
    int d=9;
    while (n) {
        if (n%10>d) return 0;
        d = n%10;
        n /= 10;
    }
    return 1;
}

int main() {
    LL n,i,ans=0;
    scanf("%lld",&n);
    for (ans=0,i=1; i<=n; i++)
        ans += step(i);
    printf("%lld\n",ans);
    return 0;
}

```

上述方法當然只能得到子題的分數，我們來看看题目的提示：「如果要計算 D 位數的階梯數字總數，我們可以依據最高位數字分成 9 類，然後根據 $(D - 1)$ 位的 9 類階梯數字總數來計算。」舉例來說明，假設 $N = 358$ 是一個三位數，要計算不大於 358 有多少階梯數字：

- 我們可以用最高位數分成 4 類：以 0, 1, 2, 或 3 開頭的三位階梯數，其中 0~2 開頭的所有階梯數都要包含，但 3 開頭的只要不大於 358 的階梯數
- 3 開頭不大於 358 的階梯數，進一步可分成 33 開頭、34 開頭、以及 35 開頭，其中 35 開頭的只要不大於 358 開頭的。
- 33 開頭的三位階梯數其實就是 3 開頭的二位數，同樣的，34 開頭的三位數就是 4 開頭的二位數。
- 35 開頭不超過 358 的就是不大於 8 的一位數，有 4 個，355~358。

從提示與上面的例子可以看出所需要的方法分兩大步驟：

- 計算一個表格 $step[i][j]$ ， $step[i][j]$ 為 j 開頭的 i 位階梯數字的個數。
- 依據輸入的數字 N ，在表格中挑選所需要累加的表格位子。

以下的表格是 4 位數以內的 $step$ 表格。

	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7	j=8	j=9

i=1	0	1	1	1	1	1	1	1	1	1
i=2	9	9	8	7	6	5	4	3	2	1
i=3	54	45	36	28	21	15	10	6	3	1
i=4	219	165	120	84	56	35	20	10	4	1

首先看表格如何計算。表格的第一列對應到 $i=1$ 位數的情形，此時根據定義除了 $j=0$ 是 0 (因為 0 不是階梯數字) 之外，其它都是 1。再看表格最右方的欄， $j=9$ 代表以 9 開頭的階梯數字，很容易知道不管幾位數，9 開頭的只有一個 99...9。對於 8 開頭的 2 位數 $\text{step}[2][8]$ ，它的第 1 位是 8，第 2 位就必須不小於 8，所以 $\text{step}[2][8] = \text{step}[1][8] + \text{step}[1][9]$ ，同理， $\text{step}[2][7] = \text{step}[1][7] + \text{step}[1][8] + \text{step}[1][9]$ ，所以我們可以推論出

$$\text{step}[i][j] = \sum_{k=j}^9 \text{step}[i-1][k]$$

如果我們進一步把 $\text{step}[i][j+1]$ 帶入計算式右方，可得

$$\text{step}[i][j] = \text{step}[i][j+1] + \text{step}[i-1][j]$$

也就是說，除了第 1 列與最後一欄之外，每一個格子都是它的右方與上方格子相加的和。所以我們可以 i 從小到大， j 從 8 到 0，很容易地計算整個需要的表格。

第二個步驟要根據輸入的數字挑選出表格中的數字來相加。以上面舉例的 $N=358$ 來看，上方表格中有標記的是前面所討論應該要加起來的格子，假設輸入 N 的每一位數儲存在 $n[]$ 陣列中而 len 是 N 的位數，在此例中也就是 $n[0]=3, n[1]=5, n[2]=8$ 而 $\text{len}=3$ ，計算過程如下：

- 從 $n[0]$ 開始看，因為 $n[0]=3$ ， $\text{len}=3$ ，所以要加 0, 1, 2 開頭的三位數，也就是 $\text{step}[3][0] + \text{step}[3][1] + \text{step}[3][2] = 135$ 。
- 現在考慮的是 3 開頭的 3 位數， $n[1]=5$ ，所以要加入的是 3 與 4 開頭的 2 位數，也就是 $\text{step}[2][3] + \text{step}[2][4] = 13$ 。
- 最後考慮的是 35 開頭的 3 位數， $n[2]=8$ ，所以要加的是 5~8 開頭的 1 位數，也就是 $\text{step}[1][5 \sim 8] = 4$ 。最後的答案就是 $135 + 13 + 4 = 152$ 。

以上大致找出了規則，但還有一個問題要處理，上面的例子 N 本身是個階梯數字，這樣處理沒錯，我們來看看 N 不是階梯數字時候的情形。例如 $N=353$ ，前兩位數的推論方式

是一樣的，但是看到第 3 位時， $n[2] = 3 < n[1]$ ，所以最後的 1 位數是不要加的。又例如 $N=317$ ，3 位數的推論依舊成立，但看第 2 位時發現 $n[1] = 1 < n[0]$ ，所以後面都不要計算了，事實上我們發現第一個往下降的位數時，後面就都不需要算了，就相當於後面都是 0 是一樣的，例如 $N=3357499$ 的答案跟 $N=3357000$ 是一樣的。

根據上面的規則可以寫成以下的範例程式。第 8~11 行以字串方式處理輸入，並將字元換成數字以便計算。第 13~22 行根據上面推出的規則計算表格，第 24~31 行根據輸入數字加總表格，請注意最後的那個 `if`，因為之前的內迴圈 `j` 只計算到 $n[i]-1$ ，所以如果 N 是階梯數字，則迴圈結束後要補一個進去。

```

00 // build step table first and then count the number
01 #include <bits/stdc++.h>
02 #define ND 20
03
04 int main() {
05     int i, j, len;
06     char n[100];
07     long long step[ND][10], total=0;
08     scanf("%s", n);
09     len = strlen(n);
10     for (i=0; i<len; i++)
11         n[i] -= '0';
12     // build table, step[i][j] for i-digits and start with j
13     for (j=1; j<10; j++)
14         step[1][j] = 1;
15     step[1][0] = 0; // 0 is not stepping number
16     // step[i][j] = sum(step[i-1][k] for k>=j)
17     // or = step[i][j+1] + step[i-1][j], suffix sum
18     for (i=2; i<=len; i++) {
19         step[i][9] = step[i-1][9];
20         for (j=8; j>=0; j--)
21             step[i][j] = step[i][j+1] + step[i-1][j];
22     }
23     // counting
24     for (j=0; j<n[0]; j++)
25         total += step[len][j];
26     for (i=1; i<len; i++) {
27         if (n[i] < n[i-1]) break;
28         for (j=n[i-1]; j<n[i]; j++)
29             total += step[len-i][j];
30     }
31     if (i == len) total++; // input is a stepping number
32     printf("%lld\n", total);
33     return 0;
34 }
35

```

以下這一節最後兩個例題維度已經不是 2D，以下我們看一個很有名的交換網路。

P-6-20. Hyper-cube path

一個維度為 n 的 hypercube 是由 2^n 個節點組成的網路，每個節點被賦予唯一一個 $0 \sim 2^n - 1$ 的編號，對於任兩個節點 i 與 j ，他們之間會有邊相連若且惟若 i 與 j 的二進位編碼恰好相差一個位元，我們對於每個節點 i 給予一個正整數的權重 $w(i)$ ，請找出編號 0 到編號 $2^n - 1$ 的一條最短路徑，使得該路徑所經過的點權重總和 (包含起點與終點) 為最大。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，第二行則是這 2^n 個節點的正整數權重 $w(0), w(1), \dots, w(2^n - 1)$ ，數字之間皆以一個空白間隔，其中 $n < 20$ 而每個權重值為非負整數不超過 100。

輸出：最大權重總和。

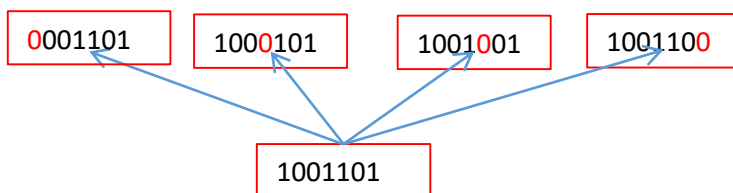
範例一輸入： 2 1 2 3 4	範例一輸出： 8
範例二輸入： 3 1 2 3 4 5 6 7 8	範例二輸出： 21

範例一說明：1 (00) $\rightarrow$ 3 (10) $\rightarrow$ 4 (11)，總和 8，括弧內為節點邊號的二進位。

範例二說明：1 (000) $\rightarrow$ 5 (100) $\rightarrow$ 7 (110) $\rightarrow$ 8 (111)，總和 21。

一個點代表一個 n -bit 的正整數，兩點可以直接相通若且惟若兩個數字只差一個 bit，我們知道起點就是 $00\dots 0$ ，終點是 $11\dots 1$ ，每走一步就是將一個 0-bit 換成 1，所謂最短路徑就是不會 0-bit 換 1-bit 後又換回 0，所以路徑的走法就是有 $n!$ 種，也就是更換 bit 的排列順序。不過不是要求路徑數，而是哪一條路徑經過的權重總和最大。其實這一題跟方格單調路徑是類似的，只是圖形比方格棋盤複雜而已。

在方格棋盤時，每一個格子的前置狀態是它的左方與上方格子，在這一題 hyper-cube (超立方體) 時，一個數字點的前置狀態是他所有為 1 的 bit 換成 0，以 1001101



為例，它的每一個 1-bit 換成 0 就是可能走到它的前置狀態，所以一共有四個 (如下圖)。

有了這個概念後，我們必須找 DP 的計算順序，這一題是 top-down 的好範例，因為計算順序比較麻煩。每一步把一個 0-bit 改成 1，所以我們應該沿著 1-bit 個數少的往多的順序來計算，也就是先算一個 1-bit 的，然後兩個 1-bit 的，...，這實在非常麻煩。我們來看看 top-down 有多簡單，以下是範例程式。遞迴函數在第 6~18 行，相同的，進來先檢查表格 `c[goal]` 是否已經算過 (≥ 0)，如果算過直接回傳。接著我們跑一個迴圈找所有前置狀態的最大值，對於每一個 bit， $(goal \& (1 \ll i))$ 檢查 goal 的第 i 個 bit 是否為 1，如果是，以 $(goal \wedge (1 \ll i))$ 把第 i 個 bit 換成 0 就是它的一個前置狀態。最大值計算完畢後記得加上該點的權重並且寫入表格 `c[]`。

```

00 // P-6-20, hypercube path, top-down
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define N 1000000
04 int c[N], w[N];
05
06 int dp(int goal, int n) {
07     if (c[goal] >= 0) return c[goal];
08     int imax = 0;
09     // check each bit, find max previous state
10     for (int i = 0; i < n; i++) {
11         if (goal & (1 << i)) { // bit-i=1
12             int k = dp(goal ^ (1 << i), n); // set bit-i to 0
13             if (k > imax) imax = k;
14         }
15     }
16     c[goal] = imax + w[goal];
17     return c[goal];
18 }
19
20 int main() {
21     int i, n, m;
22     scanf("%d", &n);
23     m = 1 << n;
24     for (i = 0; i < m; i++)
25         scanf("%d", &w[i]);
26     for (i = 0; i < m; i++) c[i] = -1; // not computed
27     c[0] = w[0]; // starting point
28     printf("%d\n", dp(m-1, n));
29     return 0;
30 }

```

這一題沒辦法寫非遞迴的形式嗎？當然是可以，但是以 graph 裡面的 BFS 比較容易，我們在範例程式中有放一支這樣的寫法，請有興趣的人自己去看。

下一題也是 top-down 的好例子。

P-6-21. 刪除邊界 (APCS201910)

一個矩陣的第一列與最後一列以及第一行與最後一行稱為該矩陣的四條邊界線，如果某一條邊界線的內容都是相同元素，則可以刪除該邊界線。如果一條邊界線的內容不完全相同，你可以修改某些格子的內容讓邊界線的內容變成相同後，再刪除該邊界線。矩陣在刪除一條邊界線後，還是一個矩陣，但列數或行數會減少一，本題的目標是重複執行修改與刪除邊界的動作，最後將整個矩陣刪除。輸入一個 0/1 矩陣，請計算最少要修改多少個元素才能將整個矩陣刪除。請注意：根據定義，只有一列或是只有一行的矩陣可以不需要任何修改就可以被全部刪除。

Time limit: 1 秒

輸入格式：第一行為兩個不超過 25 的正整數 m 和 n ，以下 m 列 n 行是矩陣內容，順序是由上而下，由左至右，矩陣內容為 0 或 1，同一行數字中間以一個空白間隔。

輸出：最少修改次數。

範例一輸入： 4 5 0 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 1 0	範例一輸出： 2
範例二輸入： 3 5 0 0 0 1 0 1 0 1 1 1 0 0 0 1 0	範例二輸出： 1

範例一說明：刪除最後一列 (修改 1 成 0，成本 1)，刪除最後一列 (成本 0)，刪除最後一列 (修改一個 1，成本 1)，成本總和 2。

範例二說明：刪除最右行 (修改 1 個 1，成本 1)，刪除最右行 (成本 0)，刪除第一列 (成本 0)，修改最後一列 (成本 0)，總和 1。

給一個矩陣，每次刪除一個邊界，刪除邊界的成本是該邊界線上 0 與 1 比較少的個數，要計算最小的刪除總成本，也就是刪除邊界的最好的順序。我們在第一章的 Q-1-11 曾經看到這一題，當時的要求是以遞迴的方式來解，所以矩陣的大小比較小，遞迴的時間複雜度是 $O((m+n)4^{m+n-2})$ ，因為每次有 4 個選擇，每選擇一次， $m+n$ 會減少 1。

從 DP 的想法來看，因為刪除的是邊界，需要計算的不外乎所有可能的子矩陣，也就是某一塊矩形，那麼有多少矩形呢？有 $O(mn)$ 個可能的左上角與 $O(mn)$ 個可能的右下角，所以有 $O(m^2n^2)$ 個，以 top-down 的寫法，反正稀哩糊塗的拉個四層迴圈把表格建好，然後照遞迴的方式稍微改一下就可以了，以下是範例程式，雖然程式不很短，但只是四個邊界每邊都要算一次，其實程式碼很簡單。

```
// P-6-21, top-down DP,  $O(m^2 * n^2)$ 
#include <stdio>
#define N 60
int a[N][N];
int memo[N][N][N][N]; // record the best for each rectangle
// rectangle= [left, top, right, bottom]
int cost(int l, int t, int r, int b) {
    if (memo[l][t][r][b] >= 0) // already computed
        return memo[l][t][r][b];
    int i, mcost, n0, n1, c;
    // top row
    n0 = n1 = 0; // numbers of 0 and 1
    for (i=l; i<=r; i++)
        a[t][i]? n1++ : n0++;
    mcost = cost(l, t+1, r, b) + (n1<n0 ? n1 : n0);
    // bottom-row
    n0 = n1 = 0;
    for (i=l; i<=r; i++)
        a[b][i]? n1++ : n0++; // can also in this way
    c = cost(l, t, r, b-1) + (n1<n0 ? n1 : n0);
    if (c<mcost) mcost = c;
    // left-column
    n0 = n1 = 0;
    for (i=t; i<=b; i++)
        a[i][l]? n1++ : n0++;
    c = cost(l+1, t, r, b) + (n1<n0 ? n1 : n0);
    if (c<mcost) mcost = c;
    // right column
    n0 = n1 = 0;
    for (i=t; i<=b; i++)
        a[i][r]? n1++ : n0++;
    c = cost(l, t, r-1, b) + (n1<n0 ? n1 : n0);
    if (c<mcost) mcost = c;
    memo[l][t][r][b] = mcost;
    return mcost;
}

int main() {
    int m, n, i, j;
```

```

scanf("%d%d", &m, &n);
for (i=0;i<m;i++) for (j=0;j<n;j++)
    scanf("%d", &a[i][j]);
for (int l=0;l<n;l++) for (int r=l;r<n;r++)
    for (int t=0;t<m;t++) for (int b=t;b<m;b++)
        memo[l][t][r][b] = (r==l || b==t)? 0: -1;
printf("%d\n", cost(0,0,n-1,m-1));
return 0;
}

```

那麼非遞迴的寫法呢？要計算的矩形顯然需要從小算到大，精確一點的說，寬度由小到大且高度由小到大，對於每一個寬度與高度，再枚舉所有可能的左上角位置，以下是範例程式，我們將計算一列與一行的成本寫成副程式，這樣程式比較清爽。

```

// non-recursive dp O(m^2 * n^2)
#include <stdio>
#define N 60
#define oo 4000
int a[N][N];
int dp[N][N][N][N]; // record the best for each rectangle
// the cost of boundary line (row,le) to (row,right)
int r_cost(int row, int le, int right) {
    int sum = 0, i;
    for (i=le; i<=right; i++)
        sum += a[row][i];
    i = right - le + 1;
    if (sum<=i/2) return sum;
    return i - sum;
}
// the cost of boundary line (top,col) to (bot,col)
int c_cost(int col, int top, int bot) {
    int sum = 0, i;
    for (i=top; i<=bot; i++)
        sum += a[i][col];
    i = bot - top + 1;
    if (sum<=i/2) return sum;
    return i - sum;
}
int main() {
    int m,n,i,j;
    scanf("%d%d", &m, &n);
    for (i=0;i<m;i++) for (j=0;j<n;j++)
        scanf("%d", &a[i][j]);
    int w, h, top, le; // width, height, top, le
    for (h=1; h<=m; h++) for (w=1; w<=n; w++) {
        // each possible top-le corner
        for (top=0; top+h<=m; top++) for (le=0; le+w<=n; le++) {
            if (w==1 || h==1) {
                dp[h][w][top][le]=0;
                continue;
            }

```

```

    }
    int mcost, cost;
    mcost = dp[h-1][w][top+1][le] + r_cost(top, le, le+w-1);
    cost = dp[h-1][w][top][le] + r_cost(top+h-1, le, le+w-1);
    if (cost < mcost) mcost = cost;
    cost = dp[h][w-1][top][le+1] + c_cost(le, top, top+h-1);
    if (cost < mcost) mcost = cost;
    cost = dp[h][w-1][top][le] + c_cost(le+w-1, top, top+h-1);
    if (cost < mcost) mcost = cost;
    dp[h][w][top][le] = mcost;
}
}
printf("%d\n", dp[m][n][0][0]);
return 0;
}

```

6.3. 進階題 (*)

DP 的題目很多，有些在競賽中出現的往往需要某些特殊資料結構與特別的優化技巧，這些題目形式對 APCS 來說是超出範圍的，以這份教材的原出發點，或許並不適合將他們放進來。不過看這份教材的也許有些是對競賽程式有興趣的，所以選了幾題不太複雜的放在這一節以供參考，如果覺得有些困難，可以跳過它們。

下一題是 LIS 的變形加權版。

P-6-22. 文無第一武無第二(@@) (*)

在一個群體中有 n 個成員，編號為 $1 \sim n$ ，第 i 個人的戰力為 $p[i]$ ，現在想要挑選一些人組成一個團隊擔任特別的任務。目前除了戰力外，還考量兩項能力值：程式力 $c[i]$ 與數學力 $m[i]$ ，俗話說「文無第一，武無第二」，如果挑選的人員中某一位的程式力與數學力都不如另外一位的話，他就會非常的沮喪而導致團隊總戰力受到傷害。你的目標是要挑出一個戰力總和最大的團隊，而且如果 i 與 j 都是被挑選的人的話，不允許 $c[i] < c[j]$ 且 $m[i] < m[j]$ 。

Time limit: 1 秒

輸入格式：第一行是正整數 n 。第二行有 n 個正整數，代表戰力 $p[1] \sim p[n]$ ，第三行有 n 個非負整數，代表程式力 $c[1] \sim c[n]$ ，第四行有 n 個非負整數，代表數學力 $m[1] \sim m[n]$ ，同行數字間以空白隔開。 n 不超過 $1e5$ ，戰力總和不超過 $1e9$ ，個程式力與數學力均不超過 $1e8$ 。

輸出：最大戰力總和。

範例一輸入： 5 2 4 1 3 7 1 2 3 3 4 6 7 4 5 1	範例一輸出： 15
範例二輸入： 5 1 2 3 4 9 4 5 4 5 8 3 6 4 5 7	範例二輸出： 9
範例一說明：挑選 $i=2\sim 5$ ，戰力總和 $4+1+3+7=15$ 。 範例二說明：挑選 $i=5$ ，戰力總和 9。	

如果我們將每個人的 $c[i]$ 看成 X 座標， $m[i]$ 看成 Y 座標， $p[i]$ 看成點的權重，這題就是平面上給 n 個點座標與權重，要找一些點的權重總和最大，條件是不允許某點的 XY 座標皆大於另外一點。如果我們從左往右看，就是一條左上往右下逐步遞減（非遞增）的序列，如果從右往左看，就是找一個權重最大的 LIS (longest increasing subsequence)，只是依照題意，等號是允許的（前面 LIS 那一題等號是不允許的）。

這一題比基本 LIS 難的原因主要不在排序，而是權重。在只需要考慮長度時，因為長度必然是 $1, 2, 3, \dots$ ，所以我們很容易的將長度 L 的最小可能結尾存在一個陣列 $last[L]$ 中，但在加權版本，我們要存的是以某數 t 結尾的最大權重，而且必須保持資料的單調性（結尾越大，權重越大），因此我們需要一個動態的資料結構。以下的範例程式是以 `map` 來存放這些資料，複雜度是 $O(n \log(n))$ 。

```
// max weight anti-chain, equality is allowed
// O(nlogn) DP
#include <bits/stdc++.h>
using namespace std;
#define N 100010
struct Member {
    int p, c, m;
} seq[N];
// large c first, breaking tie by small m
bool cmp(Member &x, Member &y) {
    if (x.c != y.c) return x.c > y.c;
    return x.m < y.m;
};
int main() {
    int n;
```

```

scanf("%d",&n);
for (int i=0;i<n;i++)
    scanf("%d",&seq[i].p);
for (int i=0;i<n;i++)
    scanf("%d",&seq[i].c);
for (int i=0;i<n;i++)
    scanf("%d",&seq[i].m);
sort(seq, seq+n, cmp);
int ans=0;
// find max weight nondecreasing sequence
map<int,int> M; // (m, P) largest P with ending m
M.insert({-1,0}); // as a boundary
for (int i=0; i<n; i++) {
    auto it = M.upper_bound(seq[i].m);
    it--; // seq[i] can append
    int w=it->second + seq[i].p;
    ans = max(ans,w);
    it = M.insert(it,{seq[i].m,w});
    if (it->second < w) { // when insert fail
        it->second = w;
    }
    // removing larger end but smaller weight
    it++;
    while (it!=M.end() && it->second<= w) {
        it=M.erase(it);
    }
}
printf("%d\n",ans);
auto it=M.end(); it--;
fprintf(stderr,"%d\n", it->second);
return 0;
}

```

下面是沒有利用單調性優化的版本，所以也不需要 map，複雜度是 $O(n^2)$ 。

```

// max weight anti-chain, equality is allowed
// O(n^2) DP without monotonic
#include<bits/stdc++.h>
using namespace std;
#define N 100010
struct Member {
    int p, c, m;
} seq[N];
// small c first, breaking tie by large m
bool cmp(Member &x, Member &y) {
    if (x.c != y.c) return x.c < y.c;
    return x.m > y.m;
};
int main() {
    int n;
    scanf("%d",&n);

```

```

for (int i=0;i<n;i++)
    scanf("%d",&seq[i].p);
for (int i=0;i<n;i++)
    scanf("%d",&seq[i].c);
for (int i=0;i<n;i++)
    scanf("%d",&seq[i].m);
sort(seq, seq+n, cmp);
int dp[N];
// find max weight nonincreasing sequence
// dp[i]=seq[i].p + max{dp[j]: seq[j].m>=seq[i].m}
for (int i=0;i<n;i++) {
    int imax=0;
    for (int j=0; j<i; j++) {
        if (seq[j].m>=seq[i].m)
            imax=max(imax, dp[j]);
    }
    dp[i]=imax+seq[i].p;
}
printf("%d\n",*max_element(dp, dp+n));
return 0;
}

```

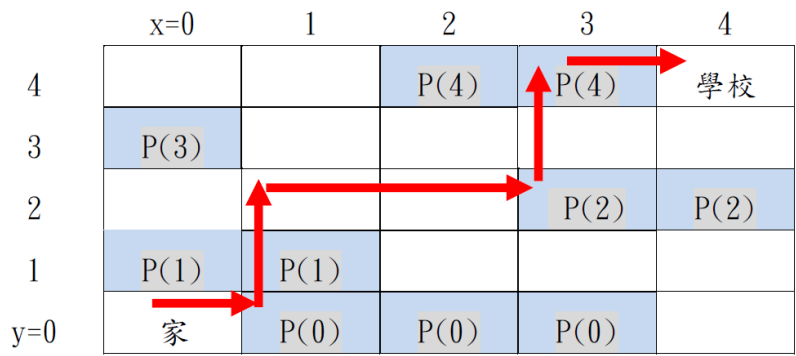
下面一題的程式其實非常簡單，只是需要一點思考，也需要使用動態的資料結構，我們把它列作習題，但給予提示。

Q-6-23. 直升機抓寶 (**) (105 高中全國賽)

小智的學校在天空之城，他每天開直升機上學，現在他想要規劃一條路徑以便在從家裡到學校的路上可以抓到最多的寶貝，請你寫個程式幫助他。

我們將學校與他家之間所有的位置劃分成 $N \times N$ 的格子，每個格子以坐標 (x, y) 表示，其中 x 代表水平距離， y 代表高度，而小智家的坐標在 $(0, 0)$ 的位置，學校在 $(N-1, N-1)$ 的位置，由於他的父母不准他上學途中貪玩繞路，直升機被設定成每次只能向前或向上一格，也就是說，如果從 (x, y) 到 $(x+1, y)$ 或 $(x, y+1)$ ，當然，他也不可以超過 $x < N$ 且 $y < N$ 範圍，否則會無法到達學校。

小智到學校的路途上一共有 N 隻寶貝，每隻寶貝可以補獲的範圍是某特定高度而水平座標在某連續區間的格子。明確的說，對於寶貝 $P(i)$ ， $0 \leq i < N$ ，要捕抓到 $P(i)$ ，小智必須經過下列座標之一： $\{(x, y) \mid S(i) \leq x \leq T(i) \text{ and } y=i\}$ ，其中所有的 $S(i)$ 與 $T(i)$ 都已經透過抓寶雷達得到資料了。



上圖是一個 $N=5$ 的例子，藍色區塊顯示可以捕抓到寶貝的地方，請注意，每一個寶貝都是在一個水平連續區間。紅線所顯示的路徑是一條合乎規定的路徑，因為他每一步都只有向右或向上，沿這一條路徑可以捕抓到四隻寶貝，是所有可能路徑中可以捕抓到寶貝數最多的。

Time limit: 1 秒

輸入格式：第一行是座標範圍的 N ，接下來 N 行，每一行有兩個整數 $S(i)$ 與 $T(i)$ ，依序是 $i=0,1,...N-1$ ，其中 $0 \leq S(i) \leq T(i) < N \leq 2.5e5$ 。

輸出：最多可能抓到的寶貝數。

範例一輸入： <pre>5 1 3 0 1 3 4 0 0 2 3</pre>	範例一輸出： <pre>4</pre>
範例二輸入： <pre>2 1 1 0 0</pre>	範例二輸出： <pre>1</pre>

如果我們從下往上一步一步看，令 $p[x]$ 是當時走到 x 座標 x 能抓到的最多寶貝數量，顯然 $p[]$ 是一個非遞減陣列。那麼其實我們要做的很簡單：在高度 i 時，讀取一個區間 $[s,t]$ ，對於 $p[s] \sim p[t]$ ，把數字增加 1；對於 $x>t$ ，如果 $p[x]<p[x-1]$ ，則 $p[x]=p[x-1]$ ，也就是做 prefix maximum 的動作。以下是這樣直接寫的程式，時間複雜度是 $O(n^2)$ ，這一題的重點就是如何優化，作法就列入 (困難的) 習題了，給一個提示： $p[]$ 是一個非遞減陣列，我們不一定要存 $p[]$ ，可以只存它上升的位置。

```

// touch as many intervals as possible by an increasing path
// naive method, O(N^2) method
#include <bits/stdc++.h>
#define N 250000
using namespace std;
int main() {
    int n, p[N]={0};
    scanf("%d",&n);
    for (int i=0;i<n;i++) {
        int s, t, j;
        scanf("%d%d",&s,&t);
        for (j=s; j<=t; j++)
            p[j]++;
        while (j<n && p[j]<p[j-1]) {
            p[j]=p[j-1];
            j++;
        }
    }
    printf("%d\n",p[n-1]);
    return 0;
}

```

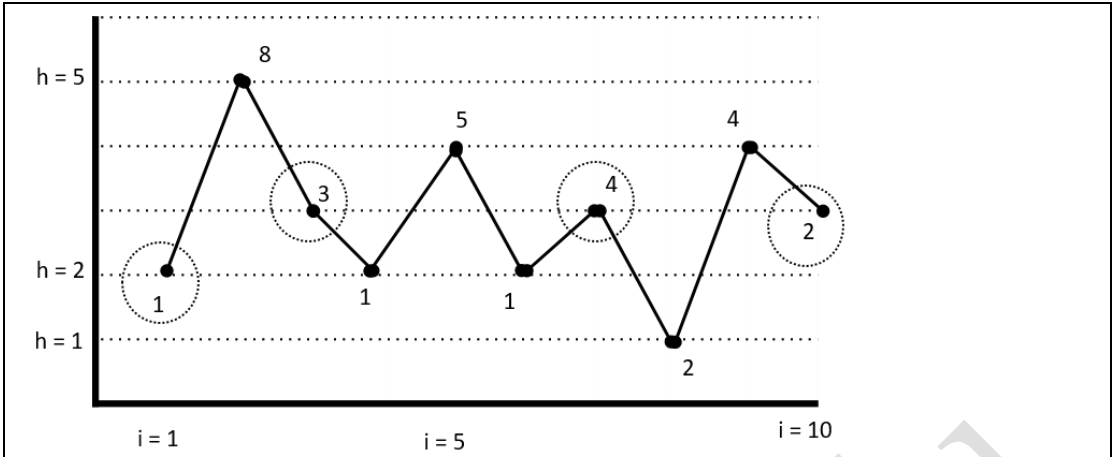
下面這一題不需要特殊的資料結構，但是思考比較困難。

Q-6-24. 隔離採礦 (???) (108 全國高中賽)

外星採礦隊在烏邦圖星球已經探勘到某種礦物的礦脈，這條礦脈是一個筆直的直線，且一共有 n 個可以挖礦井的地點，這些地點就稱為可開採點。所有的可開採點由 1 到 n 依序編號，第 i 個可開採點的高度為 $h(i)$ 而開採價值為 $v(i)$ 。

現在希望能找出某些可開採點來實際進行鑽井挖礦，因為安全因素，任兩個礦井之間必須有另外一個開採點，其高度超過這兩個礦井。身為外星採礦隊的程式設計師，你的目標是計算出最大的開採價值總和。

下圖是一個 $n=10$ 的例子，圖中各點的高度為 y 軸座標，而開採價值則標註於各點旁邊。如圖所示，各點的高度依序是 $h = (2, 5, 3, 2, 4, 2, 3, 1, 4, 3)$ ，而開採價值則依序是 $v = (1, 8, 3, 1, 5, 1, 4, 2, 4, 2)$ ，也就是說 $h(1)=2$, $h(2)=5$, ..., $h(10)=3$ ，而 $v(1)=1$, $v(2)=8$, ..., $v(10)=2$ 。在此例中，最佳的開採方式是圖中被圓圈所圈起來的四個點，所能獲得的最大開採價值總和為 $1+3+4+2=10$ 。你可以看到，任兩個實際開採點之間都有一個高度更高的點。



Time limit: 1 秒

輸入格式：每筆測資的第一行是一個正整數 n ， $n \leq 1e6$ ，代表可開採點的數量。第二行是 n 個非負整數，依序代表每一點的高度 $h(1), h(2), \dots, h(n)$ ；第三行是 n 個正整數，依序代表每一點的開採價值 $v(1), v(2), \dots, v(n)$ 。每一行相鄰數字間以一空白間隔。每一點的高度不會超過 $1e9$ ，價值不會超過 $1e5$ 。

輸出：輸出為一整數，代表最大的開採價值總和。每筆測資的答案不超過 $2e9$ 。

範例一輸入： <pre>10 2 5 3 2 4 2 3 1 4 3 1 8 3 1 5 1 4 2 4 2</pre>	範例一輸出： <pre>10</pre>
範例二輸入： <pre>5 3 3 3 3 3 5 3 2 3 4</pre>	範例二輸出： <pre>5</pre>

這題有很多解法，最好的解法是 $O(n)$ 的 DP，但是轉移式比較不容易想，另外也有幾種 $O(n \log(n))$ 的解法。

下面這一題其實是赫赫有名的經典題 TSP(travelling salesperson problem)。

Q-6-25. 貨郎問題 (QQ) 有一個賣貨郎要旅行 n 個城市做生意並且回到他的家，由於路途遙遠，他希望走的路程總和越短越好，希望你幫他規劃最短路線。這 n 個城市由 $0 \sim n-1$ 編號，他的家在

編號 m 的城市。對任意兩個城市 i 與 j ，已知兩者之間的距離是 $d[i][j]$ ，而且繞經第三地的路程絕對不會更短。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 m 。接下來 n 行每行 n 個非負整數是矩陣 $d[i][j]$ 的內容，順序由上而下，由左而右， i 從 $0 \sim n-1$ ， j 從 $0 \sim n-1$ 。 $n \leq 16$ ，假設 $d[i][j]=d[j][i]$ ， $d[i][i]=0$ ，且 $d[i][j] \leq 100$ 。

輸出：最短總路程。

範例一輸入： <pre> 4 0 0 1 1 2 1 0 1 2 1 1 0 1 2 2 1 0 </pre>	範例一輸出： <pre> 5 </pre>
範例二輸入： <pre> 4 2 0 2 1 1 2 0 1 1 1 1 0 2 1 1 2 0 </pre>	範例二輸出： <pre> 4 </pre>

範例一說明：走的順序與距離 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ ，距離總和 $1+1+1+2=5$ 。

範例二說明：走的順序與距離 $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 0$ ，距離總和 $1+1+1+1=4$ 。

一個人要把 n 個城市都走一遍再回到起點，題目中說，對任兩個城市來說，繞經第三地並不會比較近，因此單純的可以看作是一個找排列的方式，這一題的 n 到 16， $16!$ 實在太大了，所以要找更快的方法。

這一題並不需要特殊的資料結構或演算法，所以未必是 APCS 不能考的範圍，我們給予提示，作為習題。因為要繞一圈，起點根本不重要，輸入資料中給起點只是唬人的把戲，我們可以假設 0 是起點。以狀態轉移來想想看，有哪些狀態呢？對於兩個相同起點不同的路徑（尚未走完），如果它們走過的點集合一樣，最後停留的點也一樣，那麼這兩條路徑就只需要一條比較短的就可以了，例如 $P=(0, 1, 3, 7, 4)$ 與 $Q=(0, 7, 3, 1, 4)$ ，他們經過的點集合都是 $\{0, 1, 3, 4, 7\}$ 而最後停留點都是 4，所以並不需要都留下來，也就是說，我們需要考慮的狀態是所有的 (S, p) ，其中 S 是所有的子集合，

而 p 是所有可能的點，這樣的 DP 需要的複雜度還是指數，因為有 2^n 個子集合與 n 個可能的點，所以狀態數是 $O(n2^n)$ ，每一個轉移會牽涉 $O(n)$ 前置狀態，所以時間複雜度是 $O(n^2 \cdot 2^n)$ ，雖然是指數形式，但比 $O(n!)$ 要好很多。

Bangye.Wu